

Hashing

* It is a data structure technique which is used to store and search a element in a efficient manner.

Elements of Hashing:-

① Hash Table:- Simple like array which is used to store key and value pair.

② Hash function:- It is mathematical formula which is used to find the key for a given value.
key $\rightarrow$ Index!

③ key:- The key is a input function which is used to store elements in table.

④ Hash function type:-

Division method

$$h(k) = (k \% \text{size of hash table})$$

$\downarrow$ $\downarrow$
 hash given Array (or) Hash table
 key value size size

Consider hash table size as 10 and store the following into hash table using division method.

$K = 26, 14, 37, 58, 12, 61$

Sol:-
 i) $h(K) = (26 \cdot 10)$

$h(K) = 6$

ii) $h(K) = (14 \cdot 10)$

$h(K) = 4$

iii) $h(K) = (37 \cdot 10)$

$h(K) = 7$

iv) $h(K) = (58 \cdot 10)$

$h(K) = 8$

v) $h(K) = (12 \cdot 10)$

$h(K) = 2$

vi) $h(K) = (61 \cdot 10)$

$h(K) = 1$

Hash Table

0	-1
1	61
2	12
3	-1
4	14
5	-1
6	26
7	37
8	58
9	-1

Multiplicative Method

$$h(K) = \text{Size} * [K * A \bmod 1]$$

$$a \cdot b = a - b[a/b]$$

Ex:- $131.634 \cdot 1 = 131.634 - 1 * [131.634/1]$

$$= 131.634 - 1 * 131$$

$$= 0.634$$

and
using

$$h(k) = 100 * [213 * 0.618 * \text{mod } 1]$$

$$= 100 * [131.634 * \text{mod } 1]$$

$$\text{mod } 1 = 63$$

$$\text{size} = 100$$

$$k = 213$$

$$A = 0.618$$

Mid Square Method:-

- ① To square the given key value.
- ② Find the middle r digit.

Ex:-

$$k = 25$$

$$25^2 = \boxed{6}2\boxed{5} = 2$$

$$\text{Size} = 10$$

$$k = 15 \Rightarrow 15^2 = \boxed{2}2\boxed{5} = 2 \rightarrow \text{Collision value}$$

$$k = 37 \Rightarrow 37^2 = \boxed{1}3\boxed{6}9 = 6$$

Folding Method

The given key value is divided into $k_1, k_2, k_3, \dots, k_n$ based on table size. Add all the 'k' values and find hash key.

$$k = 563728 \Rightarrow \begin{array}{cc} 563 & 728 \\ \downarrow & \downarrow \\ k_1 & k_2 \end{array}$$

$$563$$

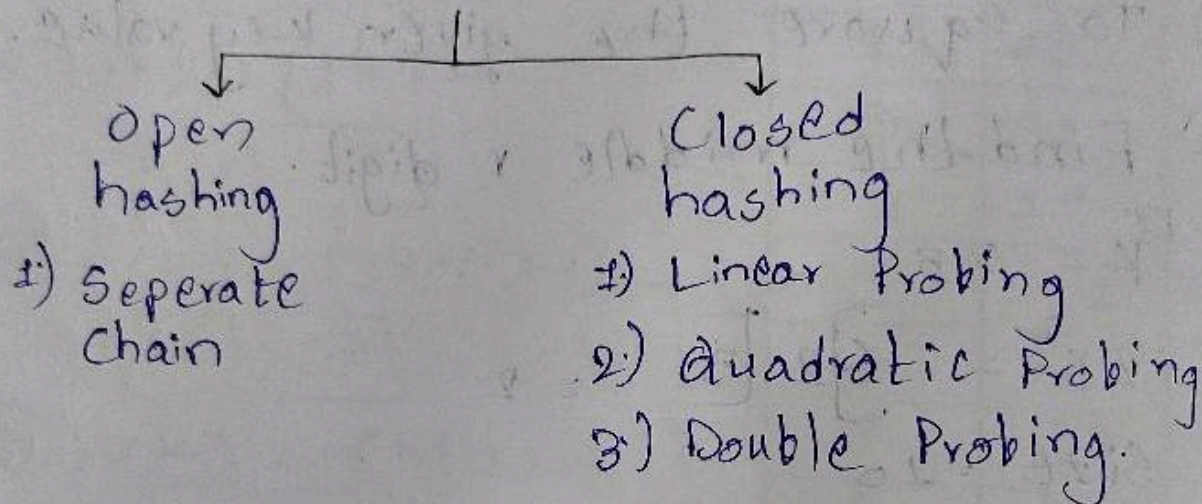
$$728$$

$$\underline{1291} \Rightarrow 129$$

Collision Avoidance:-

One or more elements are trying to access the same index in hash table is called position.

Collision



Linear Probing

Hash Function:-

$$h(K) = [(K \% \text{size}) + i] \% \text{size}$$

Q. Consider Hash Table size 10 and store the following hash table if collision occurred use linear probing method.

24, 36, 54, 68, 14, 73, 57.

$$h = [(24 \% 10) + 0] \% 10$$

$$h = 4 \% 10 = 4$$

$$h = 36 = [(36 \% 10) + 0] \% 10$$

$$= 6 \% 10$$

ing
hash

$$h=54 = [(54 \cdot 10) + 0] \cdot 10$$
$$= 4 \cdot 10$$
$$= 4 \rightarrow \text{Collision 1}$$

$$= [(54 \cdot 10) + 1] \cdot 10$$
$$= [4 + 1] \cdot 10 = 5$$

$$h=68 = [(68 \cdot 10) + 0] = 8$$

$$h=14 = [(14 \cdot 10) + 0] \cdot 10$$
$$= 4 \rightarrow \text{Collision 1}$$

$$[(14 \cdot 10) + 1] \cdot 10$$

$$= 5 \cdot 10$$

$$= 5 \rightarrow \text{Collision 2}$$

$$= [(14 \cdot 10) + 2] \cdot 10$$

$$= 6 \cdot 10 = 6 \rightarrow \text{Collision 3}$$

$$[(14 \cdot 10) + 3] \cdot 10$$

$$= 7 \cdot 10 = 7$$

$$h=73$$

$$[(73 \cdot 10) + 0] \cdot 10$$

$$= 3 \cdot 10$$

$$= 3$$

$$h=57$$

$$[(57 \cdot 10) + 0] \cdot 10$$

$$= 7 \cdot 10$$

$$= 7 \rightarrow \text{Collision 1}$$

$$[(57 \cdot 10) + 1] \cdot 10$$

$$= 8 \cdot 10 = 8 \rightarrow \text{Collision 2}$$

$$\begin{aligned} & \left[(57 \times 10) + 2 \right] \div 10 \\ &= 9 \times 10 \\ &= 9 \end{aligned}$$

0	-1
1	-1
2	-1
3	73
4	24
5	54
6	36
7	14
8	68
9	57

Quadratic Probing

$$h(k) = [(k \times \text{size}) + i * i] \div \text{size}$$

Ex:- 24, 36, 54, 68, 14, 73, 57.

$$\begin{aligned} h=24 &= [(24 \times 10) + 0 * 0] \div 10 \\ &= 4 \end{aligned}$$

$$\begin{aligned} h=36 &= [(36 \times 10) + 0] \div 10 \\ &= 6 \end{aligned}$$

$$\begin{aligned} h=54 &= [(54 \times 10) + 0 * 0] \div 10 \\ &= 4 \times 10 \\ &= 4 \rightarrow \text{Collision 1} \end{aligned}$$

$$\begin{aligned} & \left[(54 \times 10) + 1 * 1 \right] \div 10 \\ &= 5 \times 10 \\ &= 5 \end{aligned}$$

$$\begin{aligned} h=68 &= [(68 \times 10) + 0 * 0] \div 10 \\ &= 8 \times 10 = 8 \end{aligned}$$

$$h = 14 = [(14 \cdot 10) + 0 * 0] \cdot 10$$

$$= 4 \cdot 10 = 4 \rightarrow \text{Collision 1}$$

$$[(14 \cdot 10) + 1 * 1] \cdot 10$$

$$= 5 \cdot 10 = 5 \rightarrow \text{Collision 2}$$

$$[(14 \cdot 10) + 2 * 2] \cdot 10$$

$$= [4 + 4] \cdot 10 = 8 \cdot 10$$

$$= 8 \rightarrow \text{Collision 3}$$

$$[(14 \cdot 10) + 3 * 3] \cdot 10$$

$$= [4 + 9] \cdot 10$$

$$= 3$$

$$h = 73 = [(73 \cdot 10) + 0 * 0] \cdot 10$$

$$= 3 \cdot 10$$

$$= 3 \rightarrow \text{Collision 1}$$

$$[(73 \cdot 10) + 1 * 1] \cdot 10$$

$$= 4 \rightarrow \text{Collision 2}$$

$$[(73 \cdot 10) + 2 * 2] \cdot 10$$

$$= 7 \cdot 10$$

$$= 7$$

$$h = 57 = [(57 \cdot 10) + 0 * 0] \cdot 10$$

$$= 7 \cdot 10 = 7$$

$$[(57 \times 10) + 1 \times 1] \div 10$$

$$= 8 \div 10$$

$$= 8 \rightarrow \text{Collision 2}$$

$$[(57 \times 10) + 2 \times 2] \div 10$$

$$= 11 \div 10$$

$$= 1$$

0	-1
1	57
2	-1
3	14
4	24
5	54
6	36
7	73
8	68
9	-1

Double Hashing Technique:-

04/03/25

$$h(k) = [h_1(k) + i \times h_2(k)] \div \text{size}$$

$$h_1(k) = (k \div \text{size})$$

$$h_2(k) = R - (k \div R)$$

'R' is the nearest prime number to size of table.

Q. Consider hash table size 10 and store the elements in hash table if collision occurred use double hashing.
24, 36, 54, 68, 14, 73, 57.

Sol:- $h_1(24) = 24 \div 10$

$$= 4$$

$$h_2(24) = 7 - (24 \div 7)$$

$$= 4$$

$$h(k) = [4 + i \times 4] \div 10$$

$$= 4 \div 10$$

$$= 4$$

$$h(k) = h(36) = h_1(k) = (36 \cdot 10) \cdot 7 = 6$$

$$h_2(k) = 7 - (36 \cdot 7) = 6$$

$$h(36) = \cancel{36} \cdot [6 + i \cdot 6] \cdot 10 = 6 \cdot 10 = 6$$

$$h(54) = h_1(54) = (54 \cdot 10)$$

$$= 4 \rightarrow \text{Collision 1}$$

$$= [54 \cdot 10 + 1 \cdot (7 - 54 \cdot 7)] \cdot 10$$

$$= [4 + 1 \cdot (2)] \cdot 10$$

$$= 4 + 2 \cdot 10 = 6 \rightarrow \text{Collision 6}$$

$$= [54 \cdot 10 + 2 \cdot (7 - 54 \cdot 7)] \cdot 10$$

$$= [4 + 2 \cdot 2] \cdot 10$$

$$= 8 \cdot 10 = 8$$

$$h(68) = [68 \cdot 10 + 0 \cdot (7 - 68 \cdot 7)] \cdot 10$$

$$= [8 + 0] \cdot 10 = 8 \cdot 10 = 8 \rightarrow \text{Collision}$$

$$[68 \cdot 10 + 1 \cdot (7 - 68 \cdot 7)] \cdot 10$$

$$= (8 + 2) \cdot 10$$

$$= 10 \cdot 10 = 0$$

$$h(14) = [(14 \cdot 10) + 0 * (7 - 14 \cdot 7)] \cdot 10$$

$$= [4 + 0] \cdot 10 = 4 \rightarrow \text{collision}$$

$$[(14 \cdot 10) + 1 * (7 - 14 \cdot 7)] \cdot 10$$

$$= [4 + 7] \cdot 10 = 11 \cdot 10 = 1$$

$$h(73) = [(73 \cdot 10) + 0 * (7 - 73 \cdot 7)] \cdot 10$$

$$= [3 + 0] \cdot 10 = 3$$

$$h(57) = [(57 \cdot 10) + 0 * (7 - 57 \cdot 7)] \cdot 10$$

$$= [7 + 0] \cdot 10 = 7$$

0	68
1	14
2	
3	73
4	24
5	
6	36
7	57
8	54
9	

Re -
Re - h
avo

* In
ma
wo
ele
has
val

* Look

Ex:

7
int
5,

To ap
elem

→ h(3)
L.f

→ h(4)
L.f

→ h(6)

→ h(8)

* If

(3

L.

Re-hashing

05-03-25

Re-hashing is also one of the collision avoidance method.

* In the hash table we can store a maximum of 75% element. If we want to store more than 75% of element we need to resize the hash table and redistribute the values of elements.

* Load Factor = Number of entries in a hashtable / hash table size.

Ex. Consider the hash table size 7 and insert the following element into hash table, 3, 12, 14, 17, 11, 23, 6, 5, 29, 32.

To apply re-hashing to store all the elements into hash table.

$$\rightarrow h(3) = 3 \cdot 7 = 3$$

$$L.F = 1/7 = 0.1 < 0.75$$

$$\rightarrow h(12) = 12 \cdot 7 = 5$$

$$L.F = 2/7 = 0.2 < 0.75$$

$$\rightarrow h(14) = 14 \cdot 7 = 0$$

$$L.F = 3/7 = 0.4 < 0.75$$

$$\rightarrow h(17) = 17 \cdot 7 = 3$$

* If collision occurs then

$$(3+1) \cdot 7 = 4 \text{ apply linear probing.}$$

$$L.F = 4 \cdot 7 = 0.5$$

$$\rightarrow h(11) = 11 \cdot 7 = 4$$

$$(4+1) \cdot 7 = 5$$

$$(4+2) \cdot 7 = 6$$

0	14
1	
2	23
3	3
4	17
5	12
6	11

$$L.F = 5/7$$

$$= 0.71 < 0.75$$

$$\rightarrow h(29) = 23 \times 7 = 2$$

$$L.F = 6/7 = 0.8 < 0.75 \quad \times$$

After inserting 29 the Load Factor is greater than 0.75 so we need to apply for re-hashing.

Step 1:- Double the hash table size.

Step 2:- Redistribute all the element into hash table.

$$\text{Hash table size} = 2 \times 7 = 14$$

0	14
1	29
2	12
3	3
4	17
5	5
6	6
7	32
8	
9	23
10	
11	11
12	12
13	

$$\rightarrow h(8) = 3 \times 14 = 3$$

$$L.F = 1/14 = 0.07 < 0.75$$

$$\rightarrow h(12) = 12 \times 14 = 12$$

$$L.F = 2/14$$

$$L.F = 0.1 < 0.75$$

$$\rightarrow h(14) = 14 \times 14 = 14$$

$$L.F = 3/14$$

$$= 0.21 < 0.75$$

$$\rightarrow h(17) = 17 \times 14 = 3$$

$$(3+1) \times 14$$

$$= 4$$

$$4/14$$

$$= 0.28 < 0.75$$

$$\rightarrow h(11) = 11 \times 14 = 11$$

$$5/14 = 0.35 < 0.75$$

$$\rightarrow h(23) = 23 \times 14 = 9$$

$$L.F = 6/14 = 0.42 < 0.75$$

$$\rightarrow h(5) = 6 \cdot 14$$

$$= 6$$

$$L.F = 7/14 = 0.5 < 0.75$$

$$\rightarrow h(5) = 5 \cdot 14$$

$$= 5$$

$$L.F = 8/14$$

$$= 0.57 < 0.75$$

$$\rightarrow h(29) = 29 \cdot 14$$

$$= 1$$

$$L.F = 9/14$$

$$= 0.64 < 0.75$$

$$\rightarrow h(32) = 32 \cdot 14$$

$$= (4+1) \cdot 14$$

$$\cancel{L.F = 10/14} = (4+2) \cdot 14$$

$$= 6$$

$$= (4+3) \cdot 14$$

$$= 7$$

$$L.F = 10/14$$

$$= 0.71 < 0.75$$

Extendible hashing

10/08/25

Sol:-

No

* It is a dynamic hashing method which contains directory and bucket.

* No of directories can be formed based on $2^{\text{global depth}}$ (Initial global depth is 1).

* In the bucket we can store the hash values based on local depth (local depth refers to no. of L.S.B's, Initial local depth is 1).

* Fix the bucket size to store the elements or values.

* If overflow occurs in the bucket consider the following two conditions.

① If $\text{global depth} = \text{Local depth}$ then increase the global depth by 1 and increase the local depth by 1 and split the bucket and apply the re-hashing.

② If $\text{global depth} > \text{Local depth}$ then increase local depth by 1, split the buckets and apply re-hashing.

Ex:-

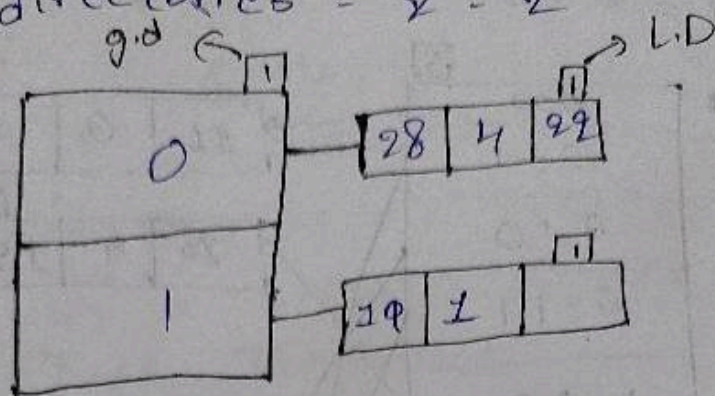
* Consider the following elements to store in hash table using extendible hashing.

28, 4, 19, 1, 22, 16, 12, 0, 5 & 7.

Consider the bucket size is 3.

Sol: $G.D = 1$

No of directories = $2^1 = 2$



28 - 11100

4 - 100

39 - 10011

1 - 001

22 - 10110

16 - 10000

12 - 1100

0 - 000

5 - 101

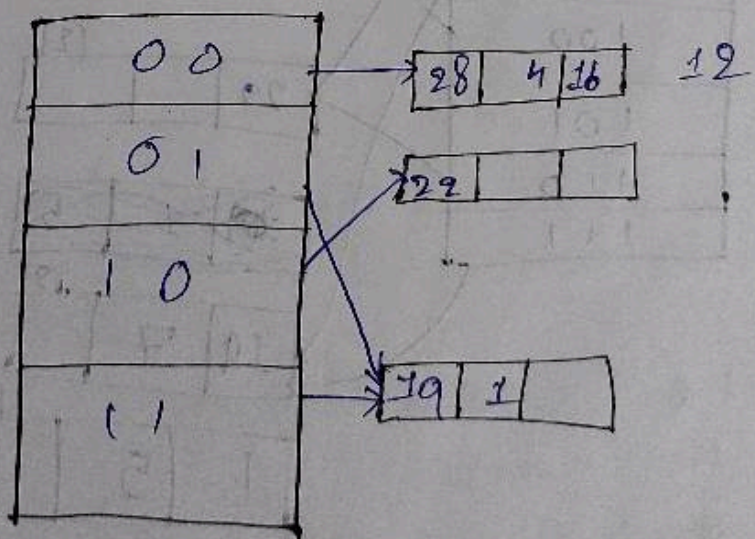
7 - 111

* We ~~are~~ cannot store 16 as it is overflow.

* $G.D = L.D$, so increase $G.D$ & $L.D$ by 1. Now

$G.D = 2, L.D = 2$

* As $G.D = 2$
No of directories = $2^2 = 4$



28 - 11100

4 - 100

22 - 10110

16 - 10000

12 - 1100

* At 12 overflow occurs.

$$G.D = 3$$

$$No \text{ of Dir} = 2^3 = 8$$

$$28 = 11100$$

$$4 = 100$$

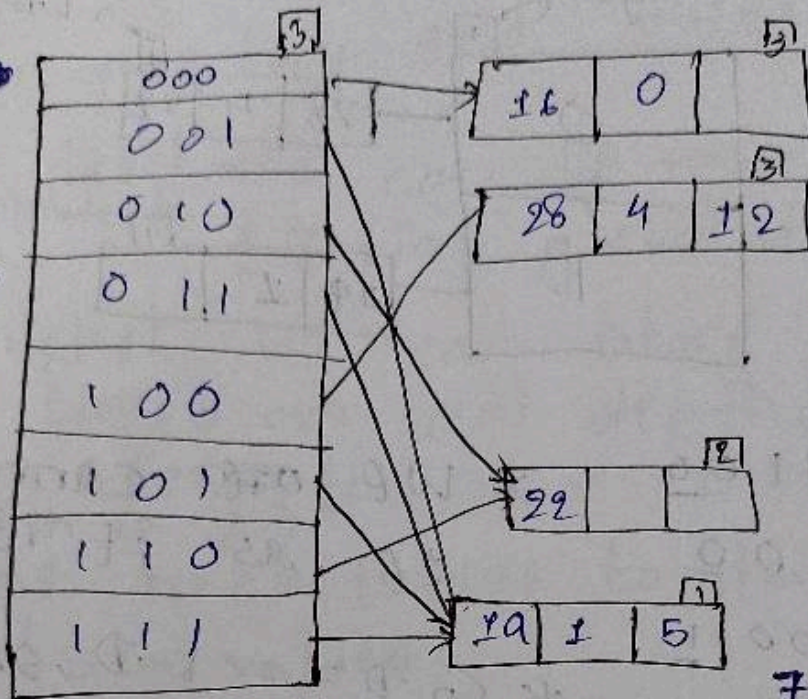
$$16 = 10000$$

$$12 = 1100$$

$$0 = 000$$

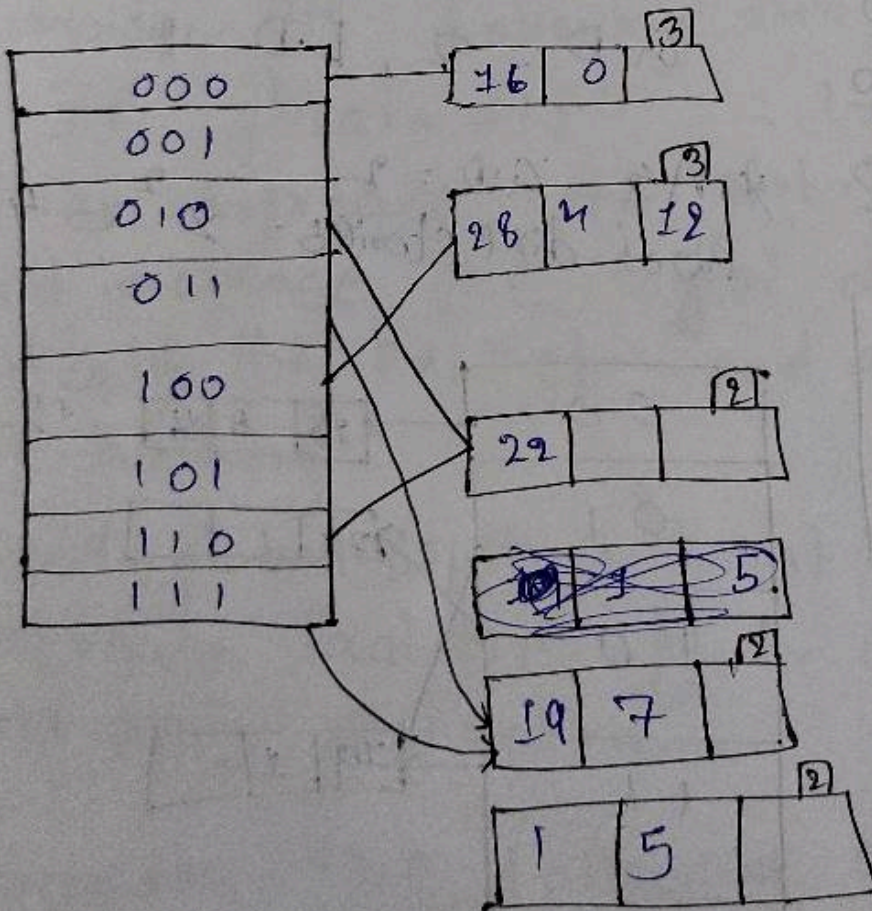
$$5 = 101$$

$$7 = 111$$



7

* At 7 overflow occurs.



$$19 = 10011$$

$$1 = 001$$

$$5 = 101$$

$$7 = 111$$

Tree

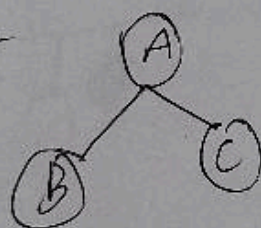
- Tree is a non-linear data structure.
- Tree is a collection of nodes which is stored in hierarchical order.

* * * Displaying (or visiting) order of nodes in a binary tree is called Binary Tree Traversal.

Types of Traversal

- | | |
|------------------------|-----------------------|
| ① In-order Traversal | (left - root - Right) |
| ② Pre-order Traversal | (Root - left - Right) |
| ③ Post-order Traversal | (left - Right - Root) |

Ex ①:-

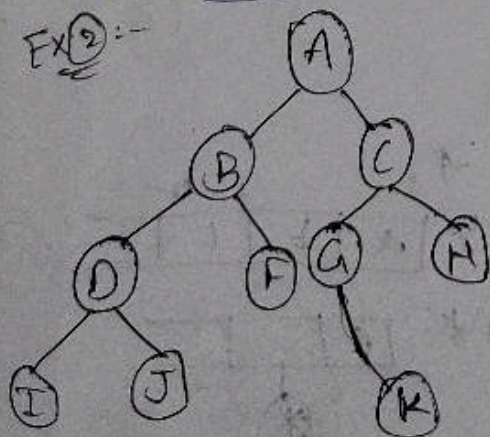


In-order:- B A C

Pre-order:- A B C

Post-order:- B C A

Ex ②:-



Inorder:- I - D - J - B - F - A - G - K - C - H

Pre-order:- A - B - D - I - J - F - C - G - K - H

Post-order:- I - J - D - F - B - K - G - H - C - A

Expression Tree

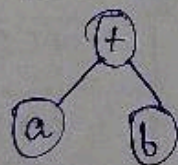
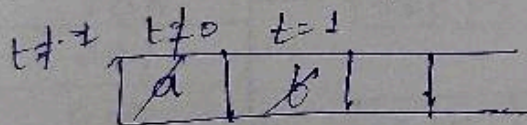
The given postfix expression is converted into expression tree using stack.

Algorithm

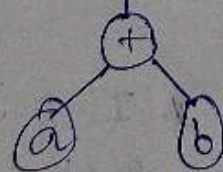
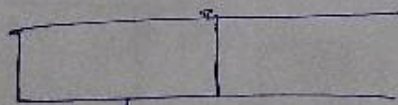
Step 1:- If char is operand push into Stack.

Step 2:- If char is operator then first pop() will be right chained & second pop() is left chained & The parent is operator then push into Stack.

Ex ①:- $a + b \Rightarrow ab +$

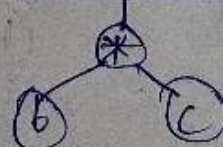
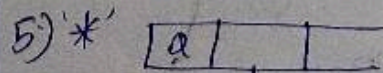
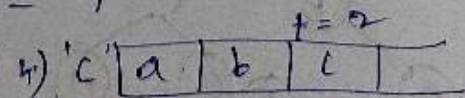
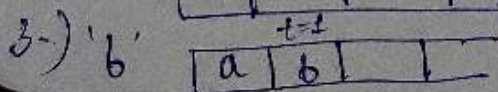
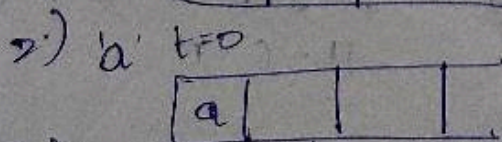
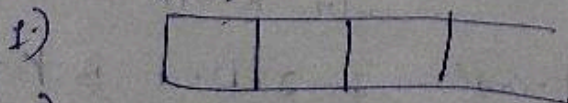


↓



Inord Ex $\Rightarrow a + b$

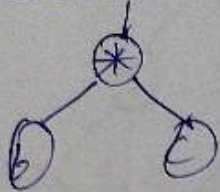
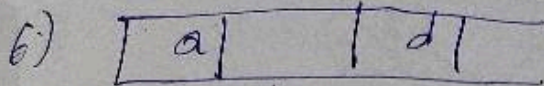
Ex ②:- $abc * d \% ef / - +$



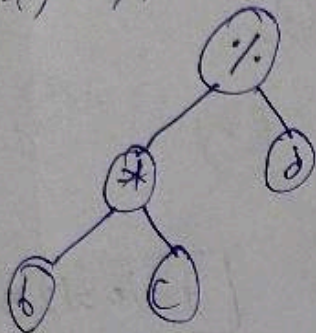
converted
it.

shinto

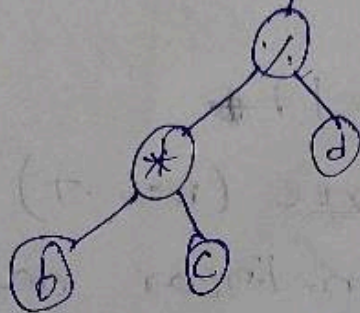
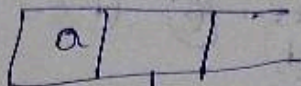
en
right
is
nt
into



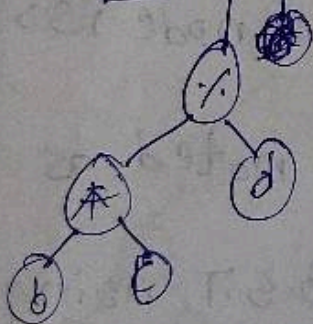
7) $'\%'$



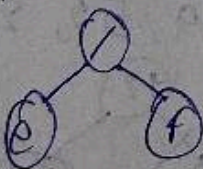
$\Rightarrow$



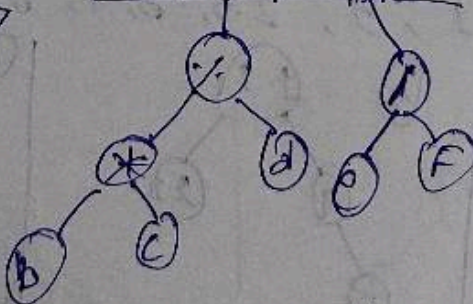
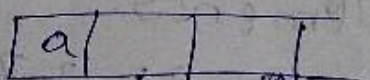
8) $'e'$



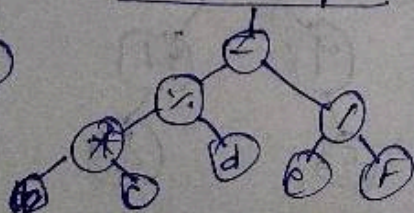
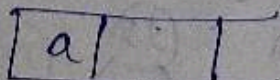
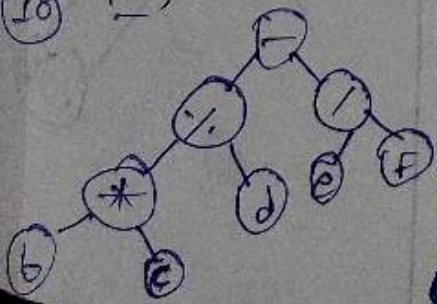
9) $'\%'$



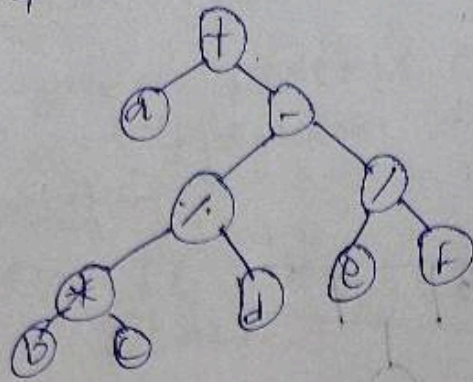
$\Rightarrow$



10) $'\%'$



11.) 4'



$a \rightarrow b * (c / d - e) / f$

Binary Search Tree (B.S.T)

11/03/25

→ BST is a non-linear data structure

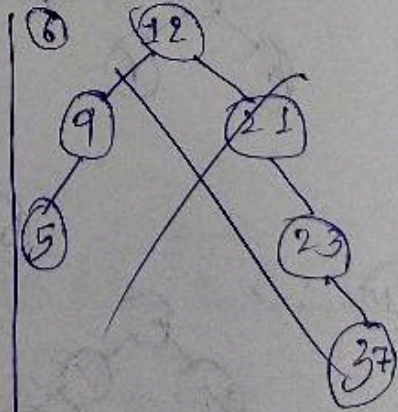
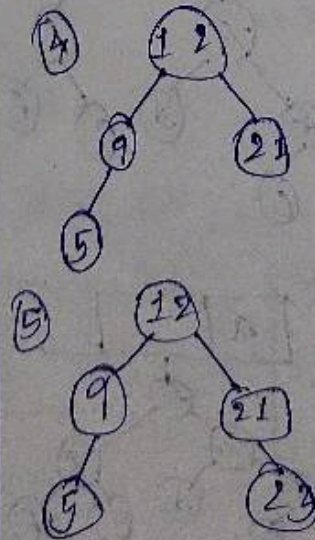
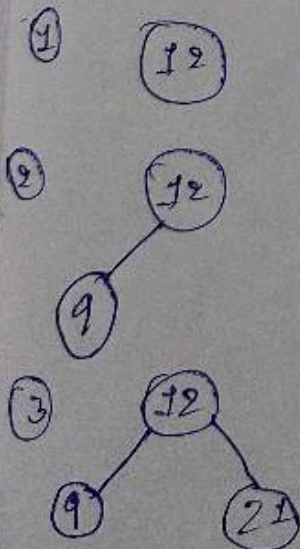
* In B.S.T every node can have maximum two child.

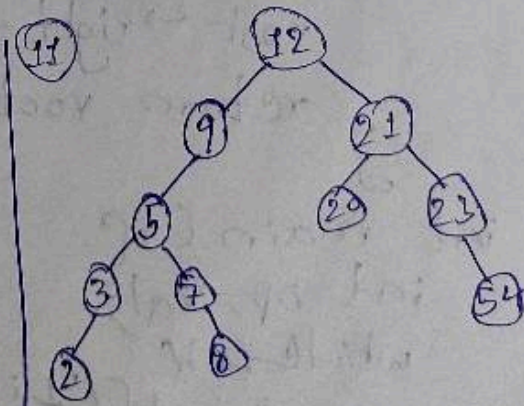
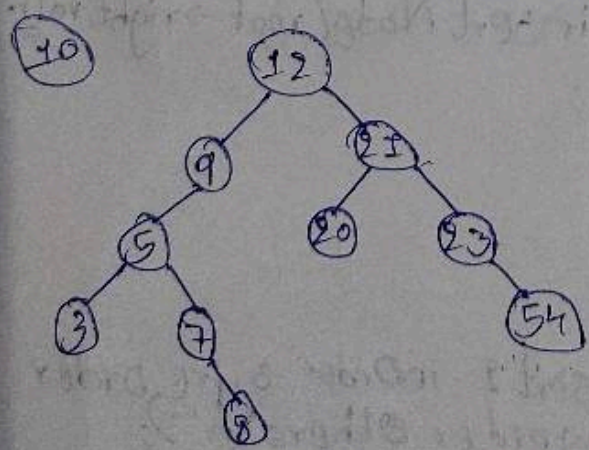
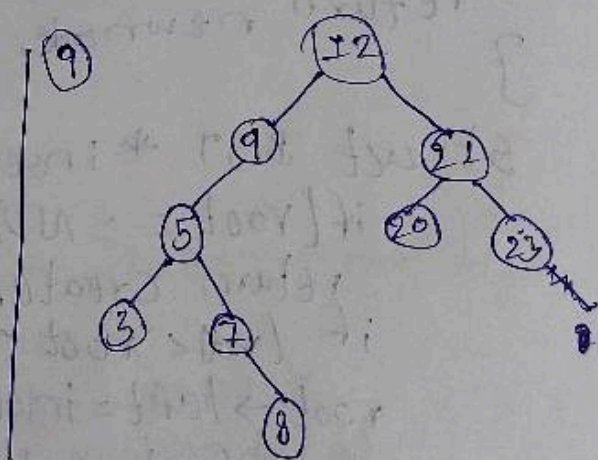
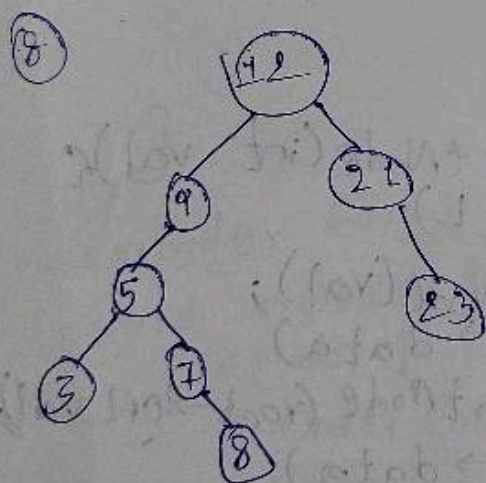
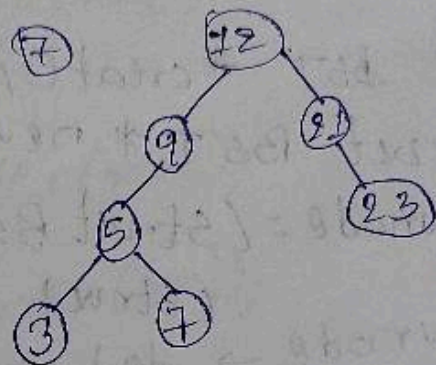
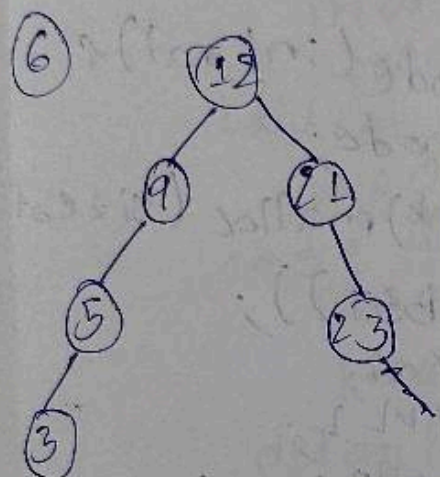
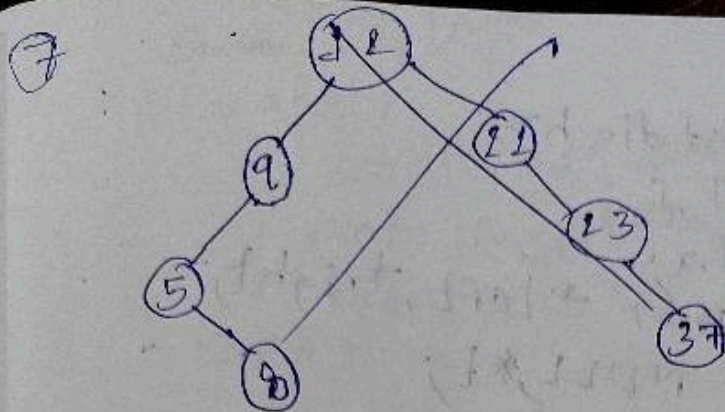
* In B.S.T left child node value is $<$ parent node & right child node is $>$ parent node.

* The first input data is noted as root node.

Q. ~~Consider~~ Construct the B.S.T using following elements.

12, 9, 21, 5, 23, 3, 7, 8, 29, 54, 2





103/25
structure
is
is
as
using
2



Code

```
#include <stdio.h>
```

```
struct BST {
```

```
    int data;
```

```
    struct BST *left, *right;
```

```
} *root = NULL, *t;
```

```
struct BST *createNode(int val) {
```

```
    struct BST *newnode;
```

```
    newnode = (struct BST *) malloc (sizeof  
        (struct BST));
```

```
    newnode → data = val;
```

```
    newnode → left = NULL;
```

```
    newnode → right = NULL;
```

```
    return newnode;
```

```
}
```

```
struct BST *insertNode(int val) {
```

```
    if (root == NULL)
```

```
        return createNode(val);
```

```
    if (val < root → data)
```

```
        root → left = insertNode (root → left, val);
```

```
    else if (val > root → data)
```

```
        root → right = insertNode (root → right, val);
```

```
    return root;
```

```
}
```

```
int main () {
```

```
    int op, val;
```

```
    while (1) {
```

```
        printf("1. insert 2. inorder 3. pre order  
            4. postorder 0. others\n");
```

```
        scanf("%d", &op);
```

Switch (op){

case 1:

scanf("%d", &val);

root = insertNode (root, val);

break;

case 2:

inOrder (root);

break;

case 3:

preOrder (root);

break;

case 4:

postOrder (root);

break;

default: printf("enter valid option");

}

}

}

Inorder function

void inOrder (struct BST *root){

if (root != NULL){

inOrder (root -> left);

printf("%d", root -> data);

inOrder (root -> right);

}

}



Preorder function

```
void preorder(struct BST *root){  
    if (root != NULL){  
        printf("%d ", root->data);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

Post order function

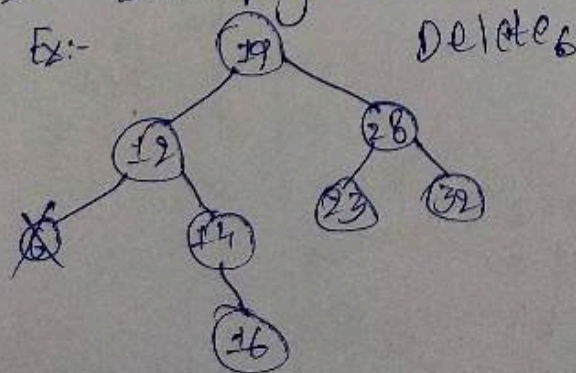
```
void postorder(struct BST *root){  
    if (root != NULL){  
        printf("%d ", root->data);  
        postorder(root->left);  
        postorder(root->right);  
        printf("%d ", root->data);  
    }  
}
```

Delete Node in BST

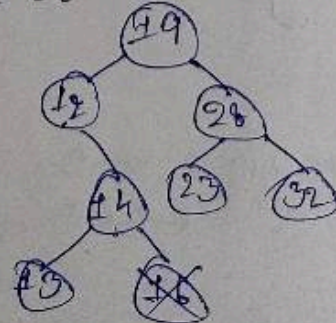
① To delete the node which is not having left child and right child (leaf node)

Sol:- Simply delete the nodes.

Ex:-



Delete 16



② To delete only

Sol:- Re

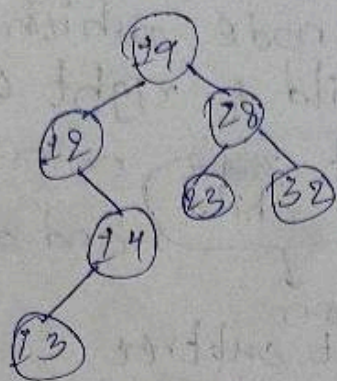
del

Sol:- Re

del

delete

delete



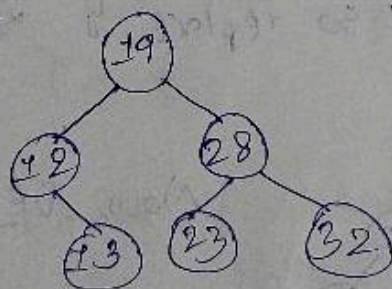
② To delete the node which is having only right child.

Sol:- Replace the right child node and delete.

③ To delete the node which is having only left child.

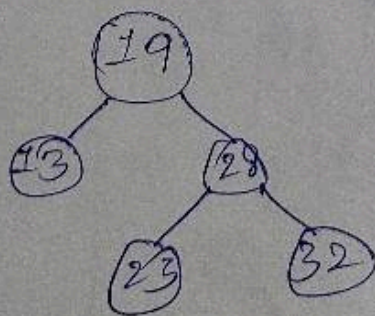
Sol:- Replace the left child node and delete.

delete 14



* We have to replace 14 with 13 & remove 14.

delete 12

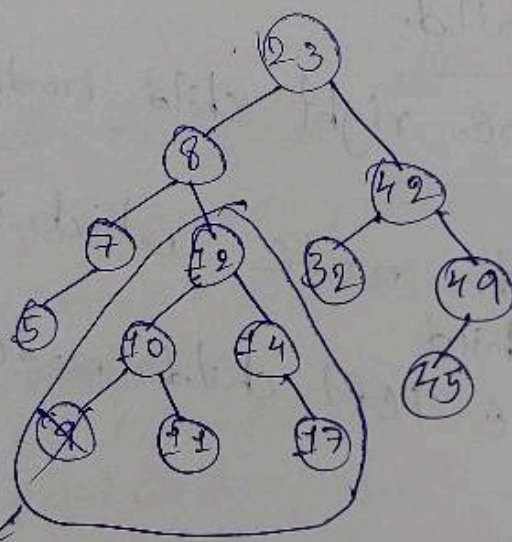


③ To delete the node which is having left child & right child.
Sol:- Replace right ~~side~~ child minimum and delete value

↓
 (or)
 right subtree

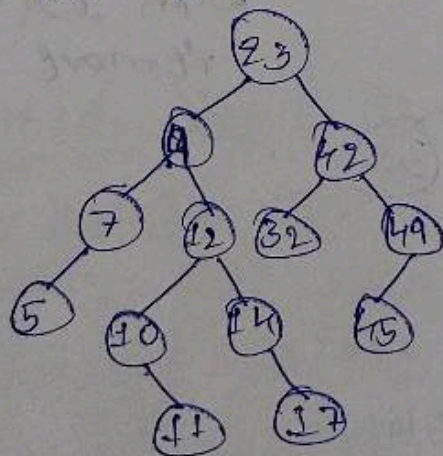
* This is also called inorder successor.

Ex:-



Delete 8

min value in right subtree so replace '8' with 9. & delete 9.



Now, Delete 23

* ~~After~~

1)

2)

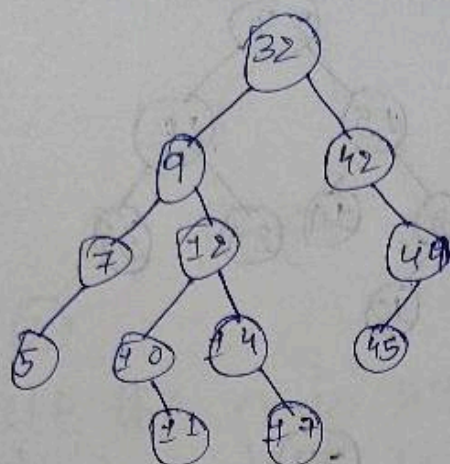
3)

15

2

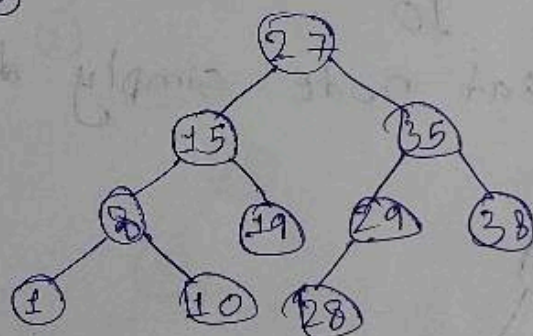
13

4

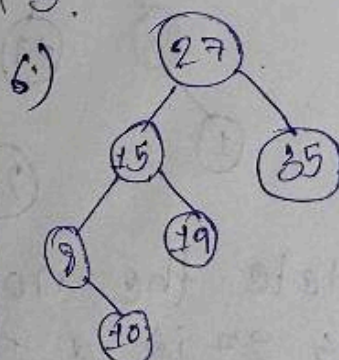
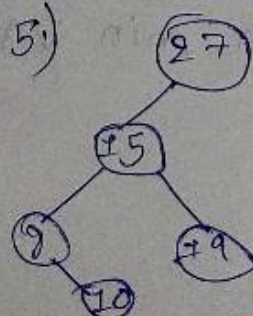
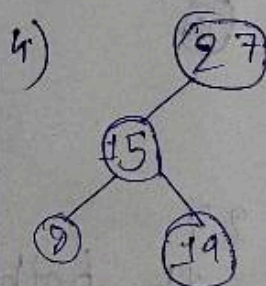
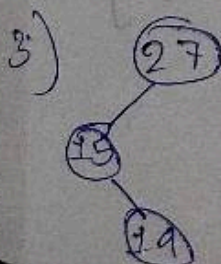
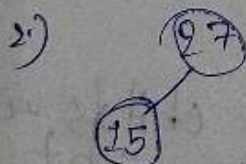


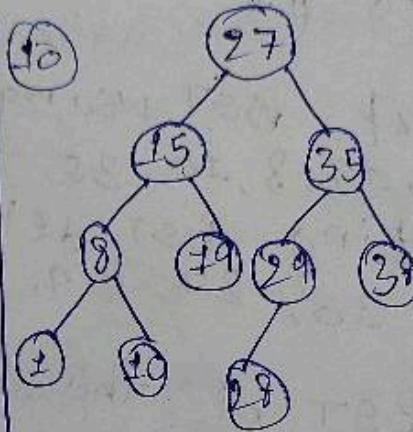
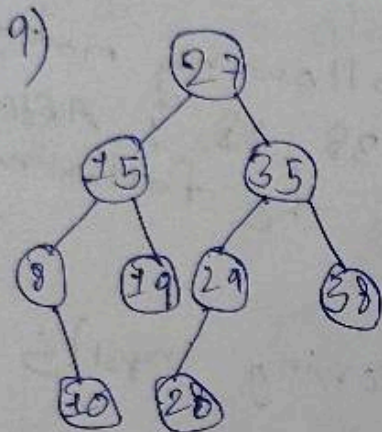
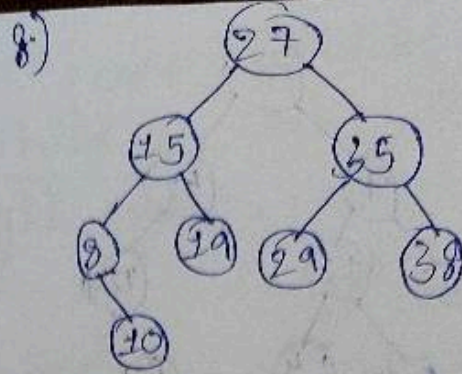
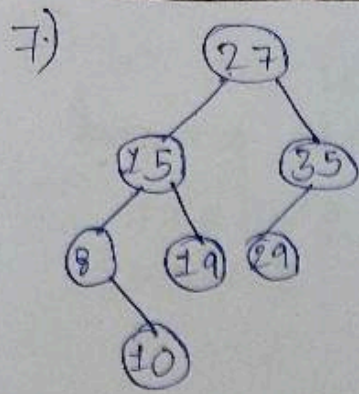
Q. Construct BST using following nodes
27, 15, 19, 8, 10, 35, 29, 38, 28, 1. After
constructing BST delete the following
nodes 10, 27, 29.

Sol:- The BST of the following inputs
is



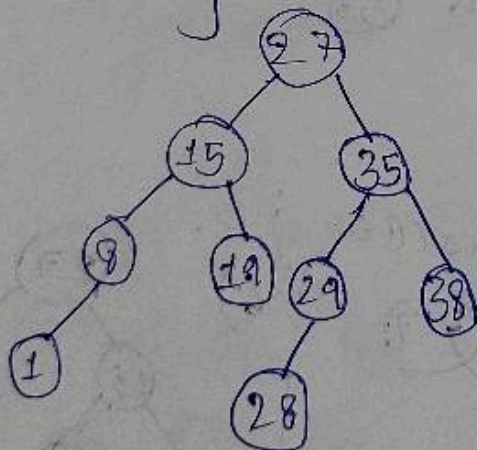
~~*After~~ The above process is.





*Delete
As it
node

*Delete the node 10.
As it is a leaf node simply delete the node.
After deleting

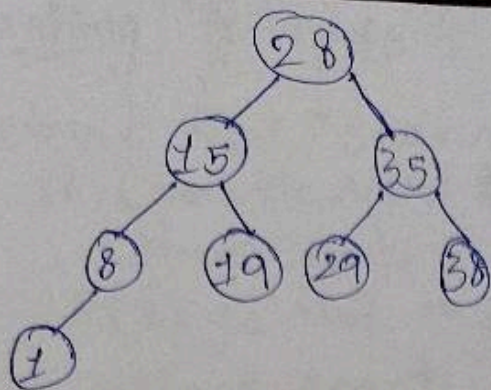


*Delete the node 27.
As 27 is a node with both the childs we have to replace with min value in right subtree.
After deleting.

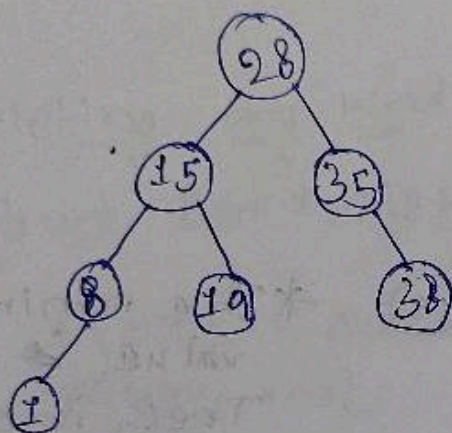
Ex-

Del

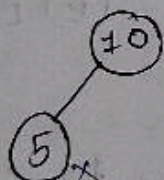
Del



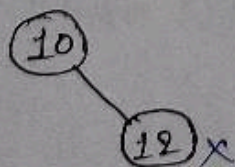
*Delete the node 29.
As it is a leaf node simply delete the node.



Ex:-



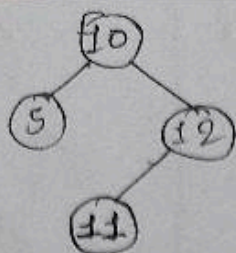
Delete 10



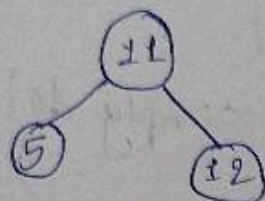
Delete 12



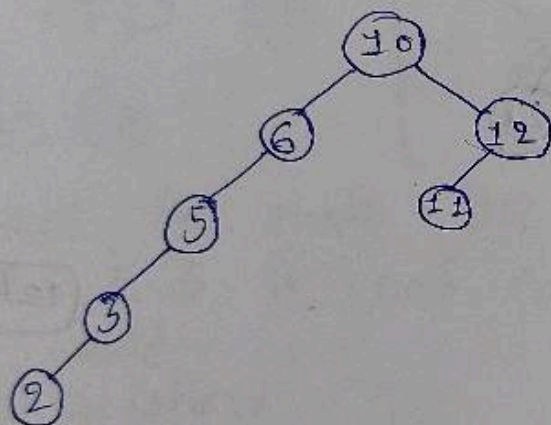
12/03/25



Delete 10



10, 12, 6, 5, 3, 2



*The minimum value is of the tree is available on left side or left subtree.

1 To Find the minimum element in tree.

```

struct BST *findmin (struct BST *root) {
    if (root->left != NULL)
        return findmin (root->left);
    else
        return root;
}
  
```

searching a node

```
struct BST *search(struct BST *root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}
```

for deleting any kind of node

```
struct BST *delete(struct BST *root, int val) {
    if (root == NULL)
        return root;
    if (val < root->data)
        root->left = delete(root->left, val);
    else if (val > root->data)
        root->right = delete(root->right, val);
    else {
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        else if (root->left == NULL) {
            t = root->right;
            free(root);
            return t;
        }
    }
}
```

```

else if (root->right == NULL) {
    t = root->left;
    free(root);
    return t;
}
else {
    t = findmin(root->right);
    root->data = t->data;
    root->right = delete(root->right, t->data);
}
}
return root;
}

```

AVL Tree (Adelson-Vskiy & Landis)

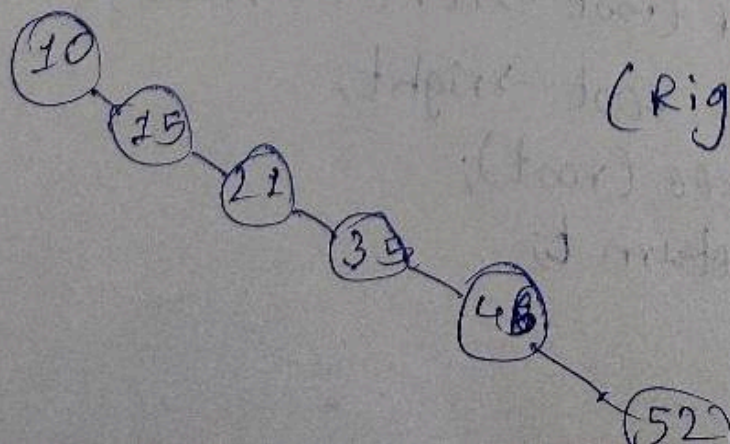
17/03/25

Drawbacks of B.T

→ While constructing B.T the input elements are in ascending order or descending order.

* For ascending order we get Right skewed
 * Descending order = Left skewed.

Ex:- 10 15 21 35 46 52



(Right-skewed).

Ex:- 1

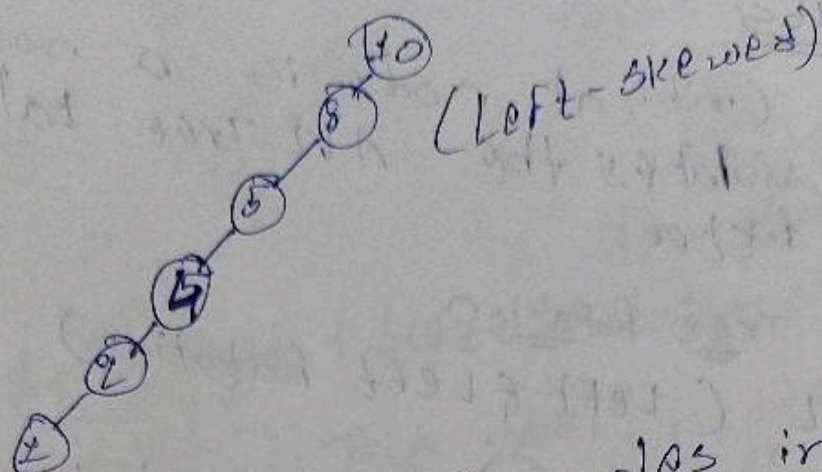
* The
 the
 O(n)
 avoid
 back

→ AV
 where
 - 1 C

* The
 to

* In
 comp

Ex: - 10 8 5 4 2 1



* The above two examples increase the time complexity of BST to $O(n)$ instead of $O(\log n)$. So, for avoiding this, we use AVL tree.

→ AVL tree is a self-balanced BST where each node balance factor is -1 (or) 0 (or) 1 .

* The balance factor of node is equal to (height of left sub-tree - height of right sub-tree).

$$b.f = H_L - H_R$$

$$-1 \leq b.f \leq 1$$

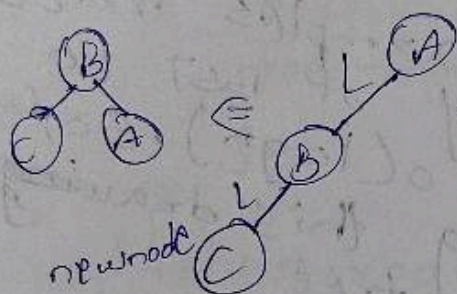
* In AVL tree the searching time complexity is always $O(\log n)$.

Construct AVL Tree using following inputs

Note:- Critical node is a node which violates the AVL Tree balance factor.

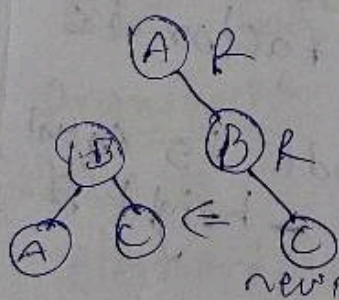
AVL Tree Rotation

① L.L (Left & Left Rotation)



⇒ The newly inserted node is left subtree of the critical nodes - solution rotate right.

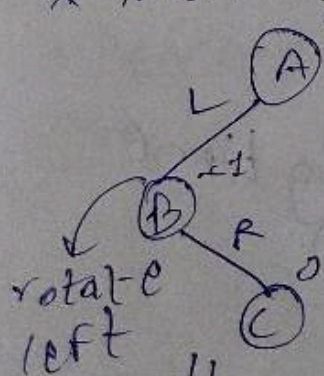
② R.R (Right & Right Rotations)



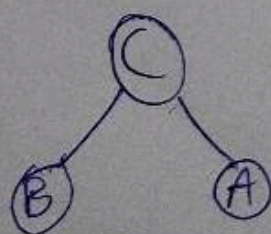
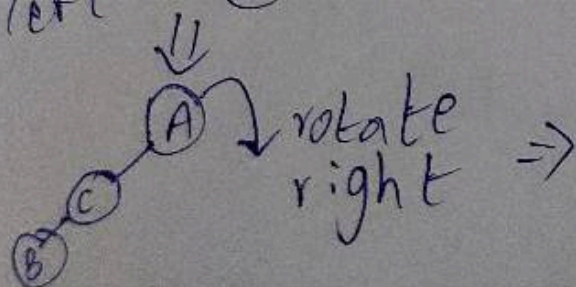
⇒ The newly inserted node is right-sub-tree of the critical nodes - Rotate left.

③ L.R (Left & Right Rotation)

* This is also called double rotation.



⇒ The newly inserted node is right subtree of the left subtree of the critical nodes.



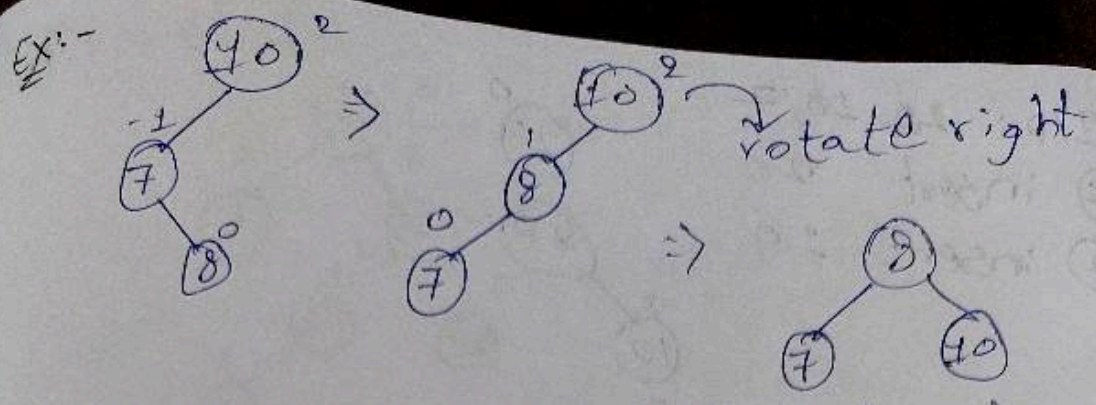
Ex:-

14

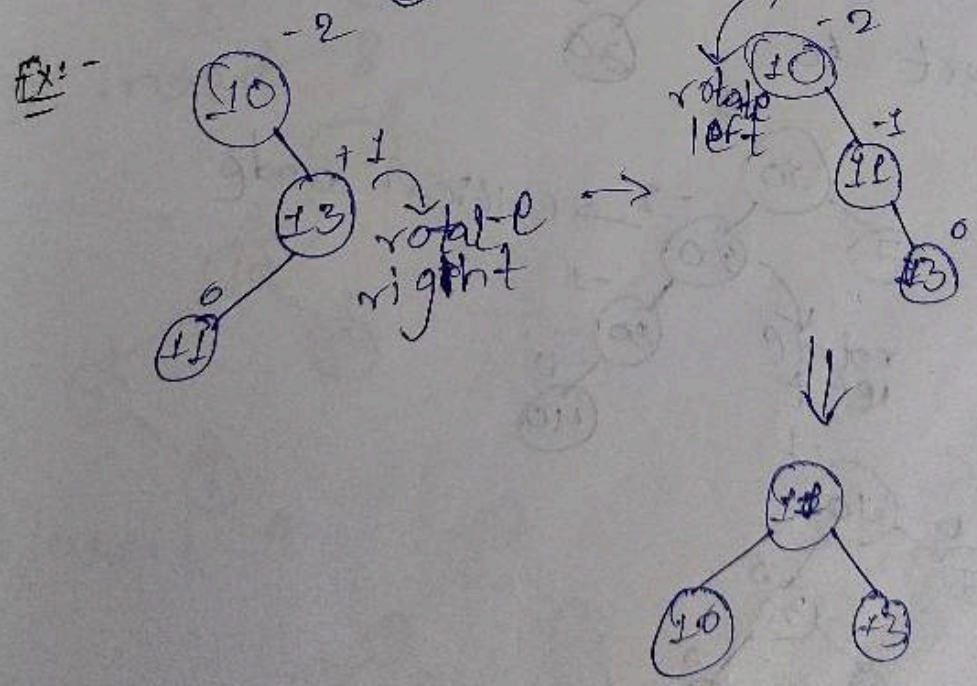
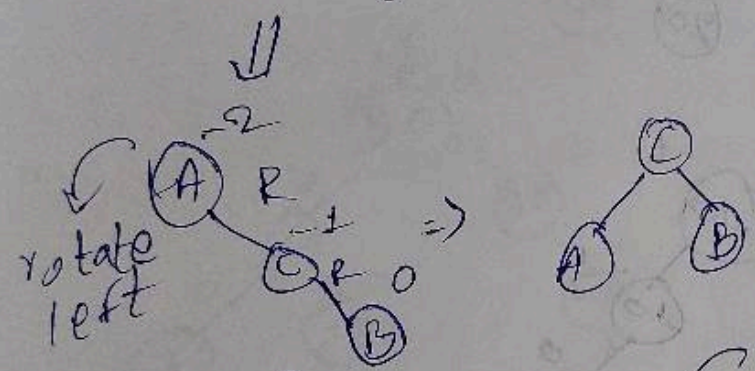
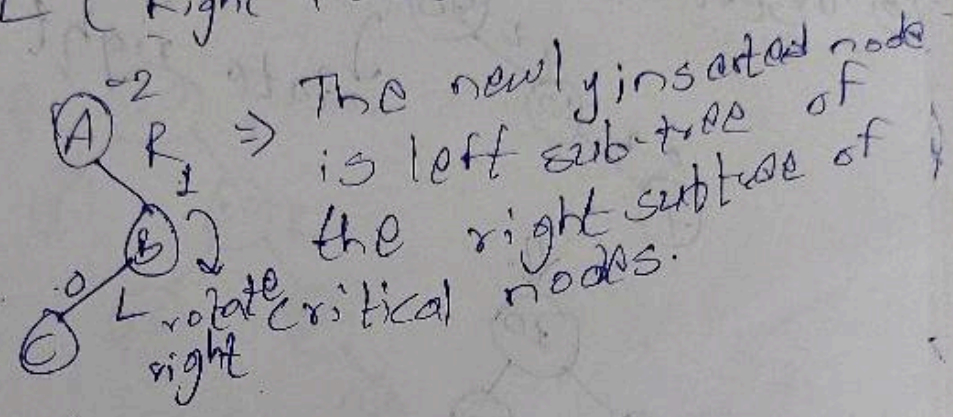
rotate left

Ex:-

ing
which
nce



④ R.L (Right & Left Rotations).

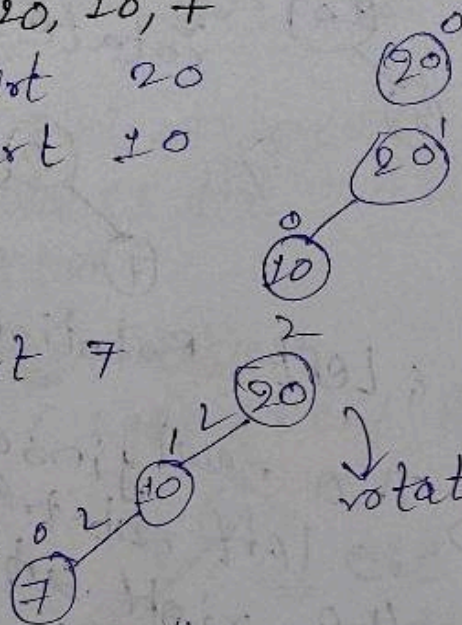


I/P: 20, 10, 7

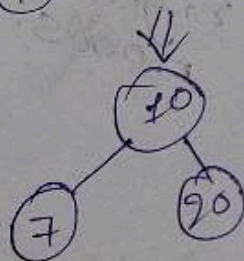
① insert 20

② insert 10

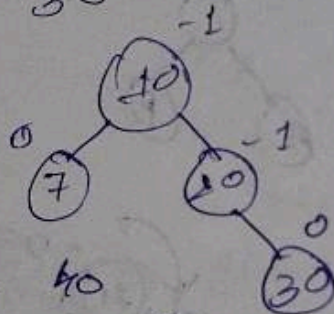
③ insert 7



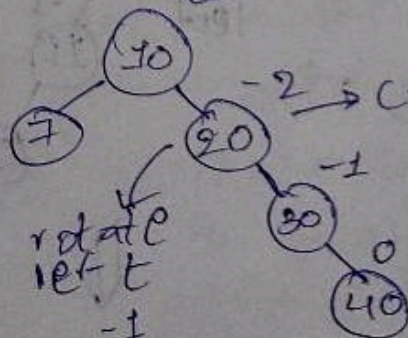
rotate right



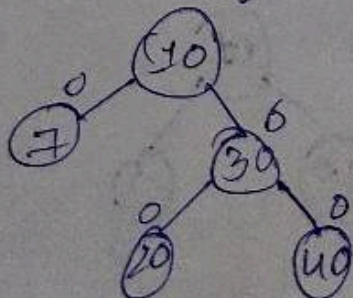
④ insert 30



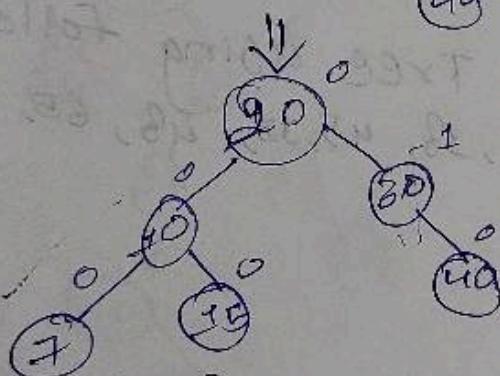
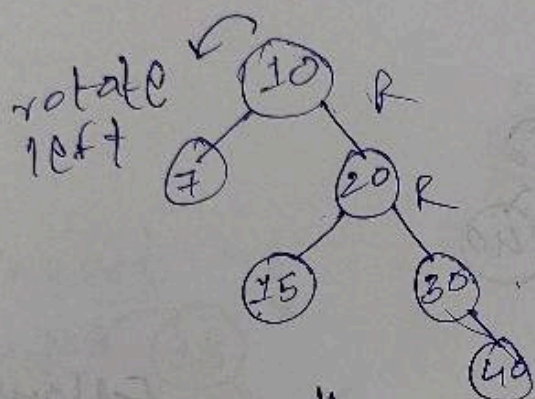
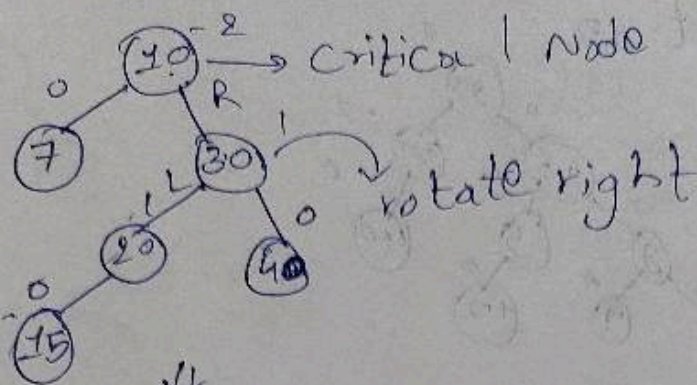
⑤ insert 40



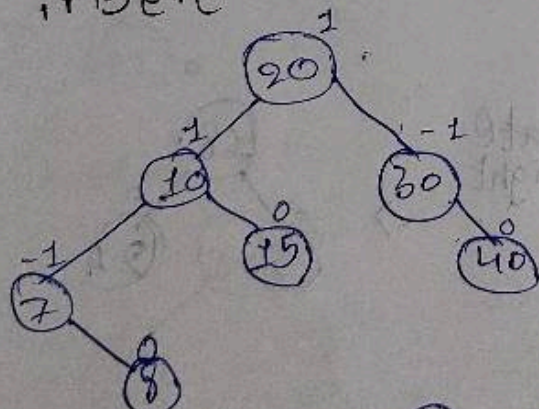
rotate left



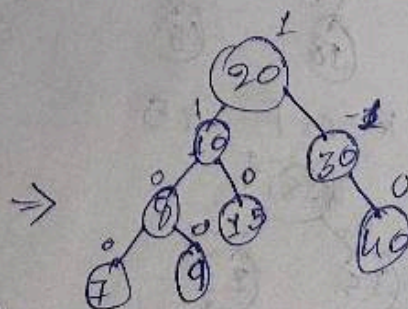
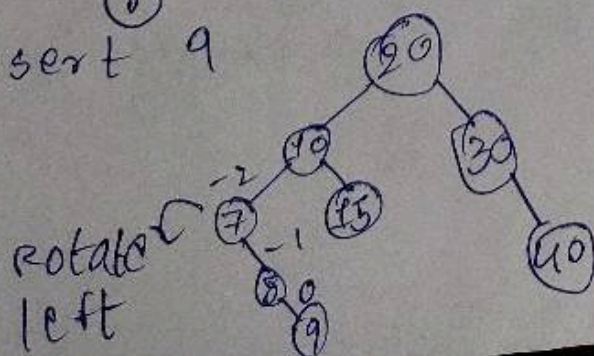
⑥ insert 15



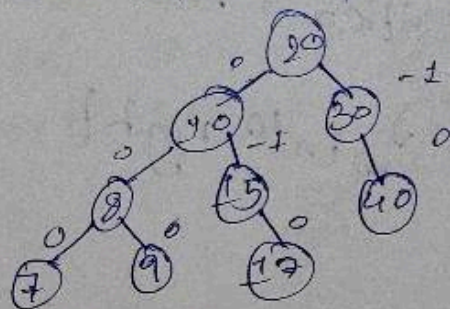
⑦ insert 8



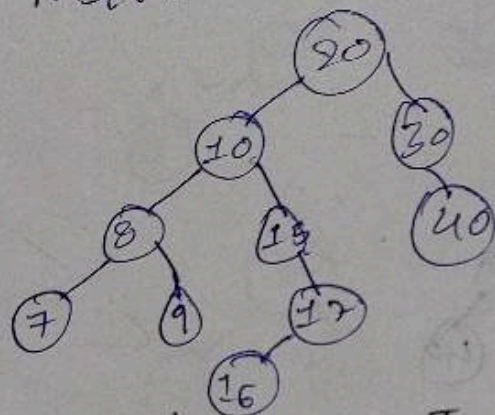
⑧ insert 9



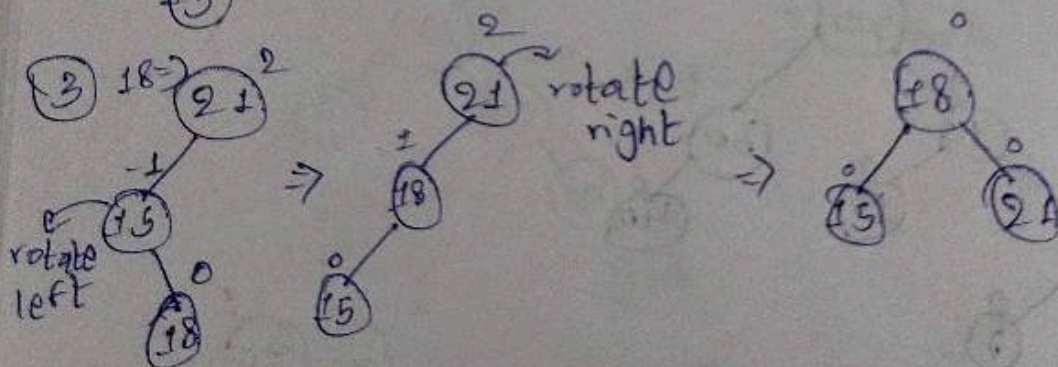
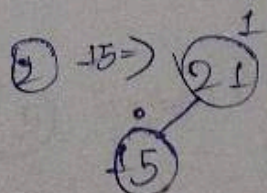
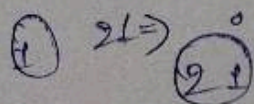
9) insert 17



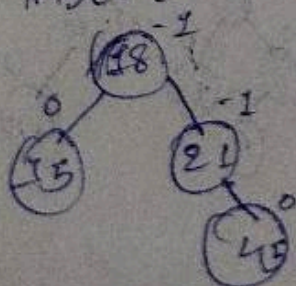
10) insert 16



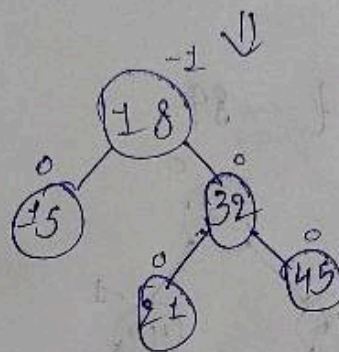
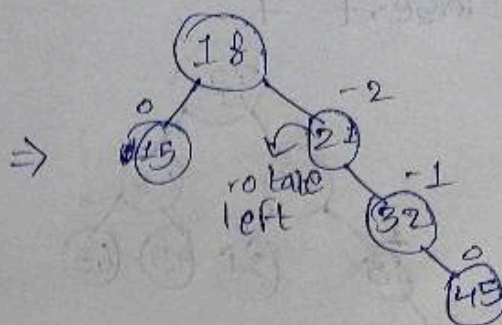
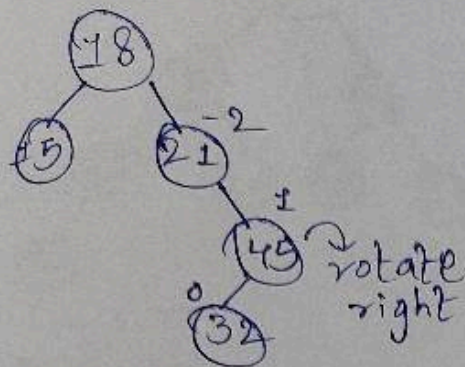
Construct AVL Tree using following nodes. 21, 15, 18, 45, 32, 48, 65, 7, 89



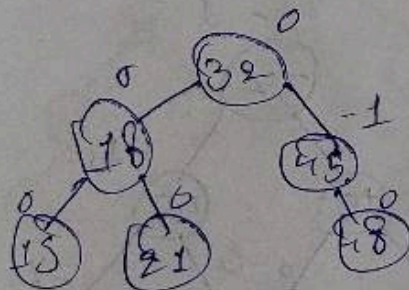
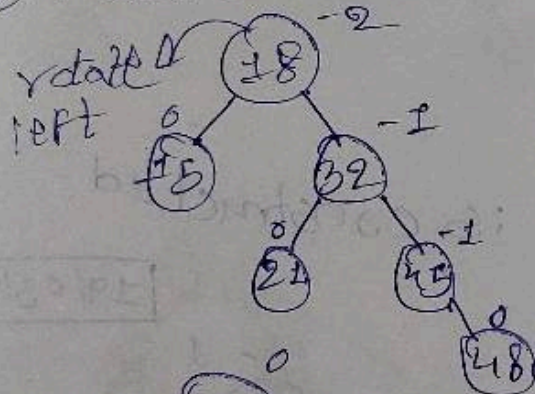
4) insert 45



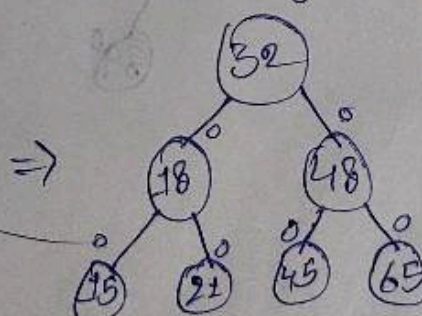
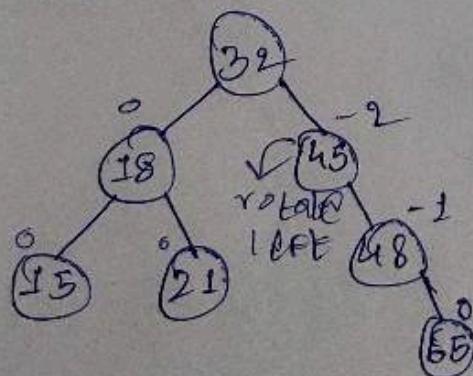
5) insert 32



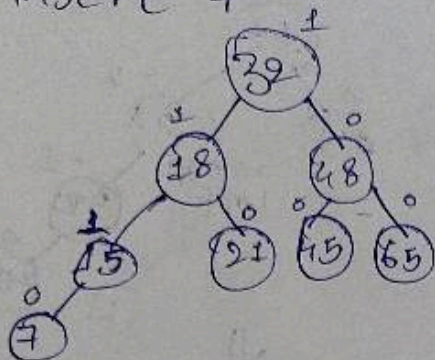
6) insert 48



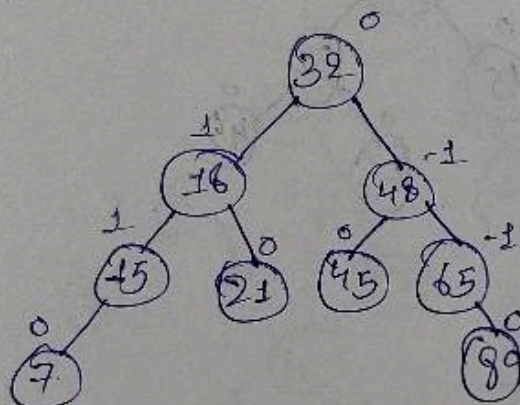
7) insert 65



⑧ insert 7



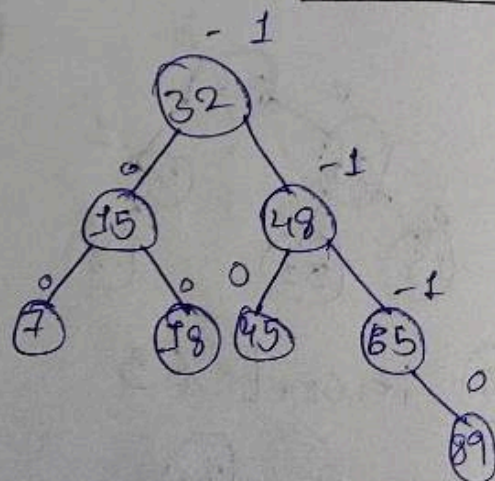
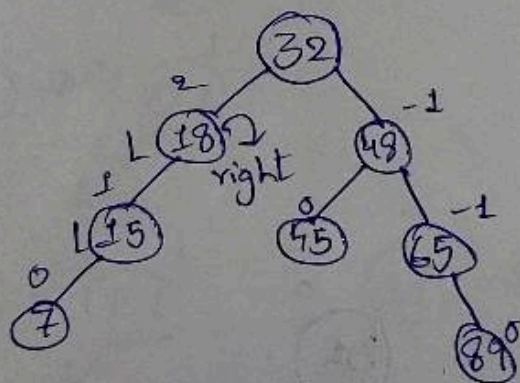
⑨ insert 89

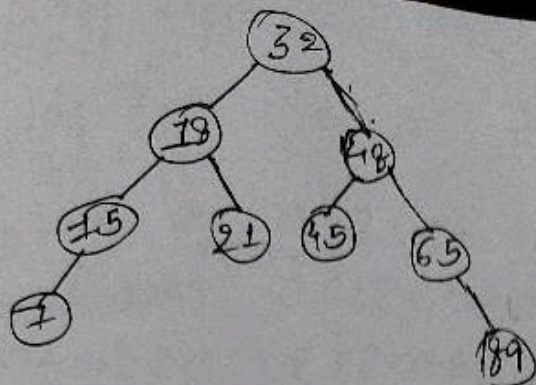


Hence the AVL Tree is constructed.

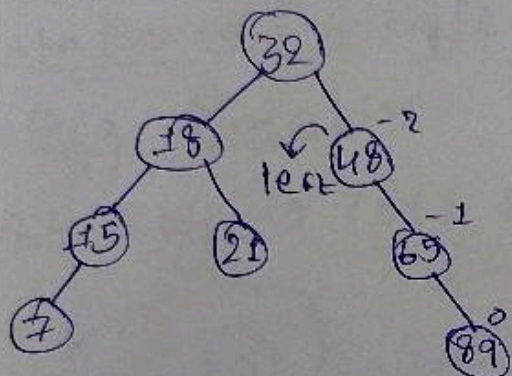
19/03/25

Delete 21

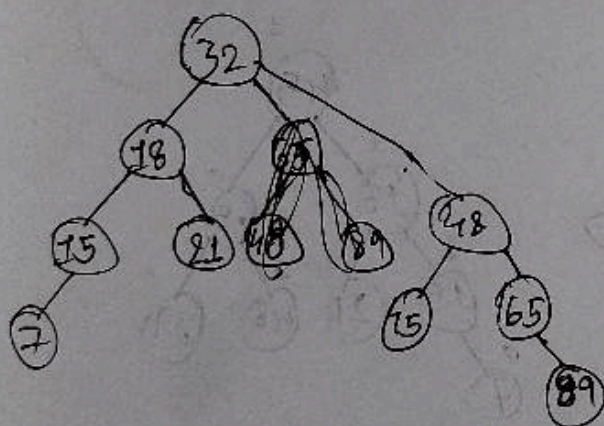
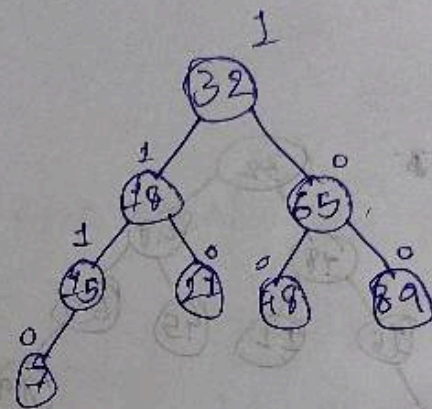




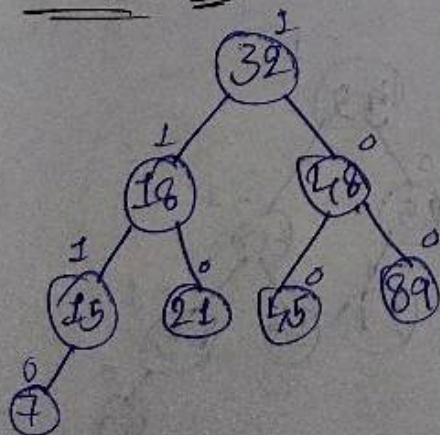
Delete 45



$\Rightarrow$

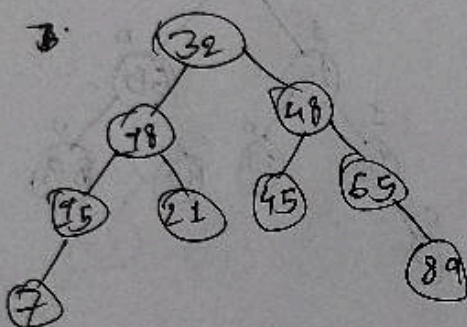
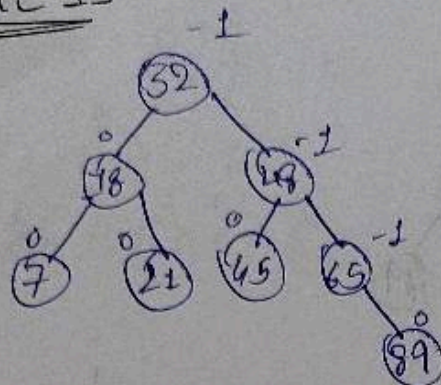


Delete 65

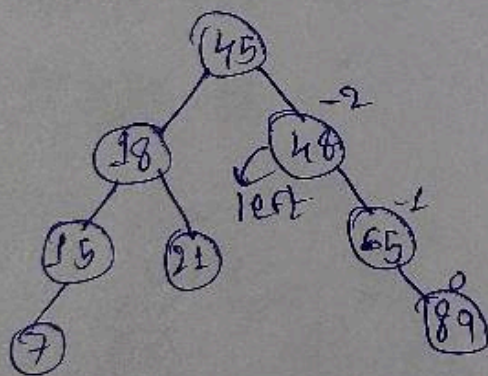


* No need of rotation.

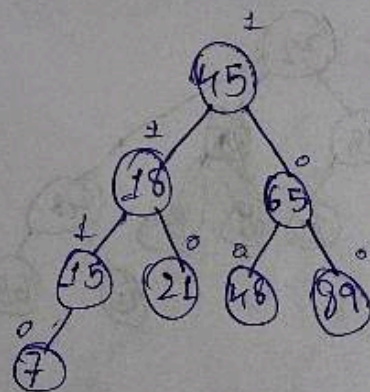
Delete 15



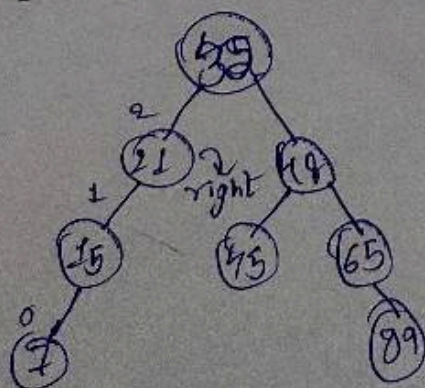
Delete 32



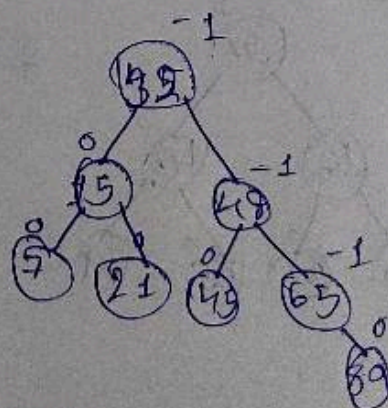
⇒



Delete 18



⇒



Splay Tree

{24/03/}

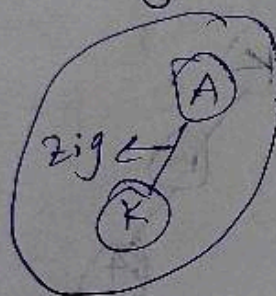
* Splay Tree is a self-adjusted BST

* Splay is a tree which brings most ~~recently~~ recently accessed element or newly inserted element into the root of the tree.

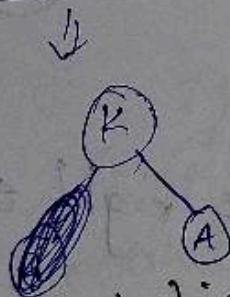
Rotations (Single)

① Zig Rotation

→ The key is left of the root.
side

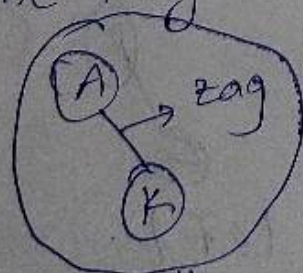


⇒ In this case we have to rotate right side.

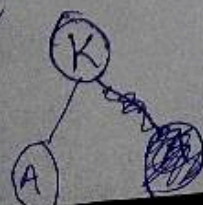


② Zag Rotation

→ The key is right side of the root



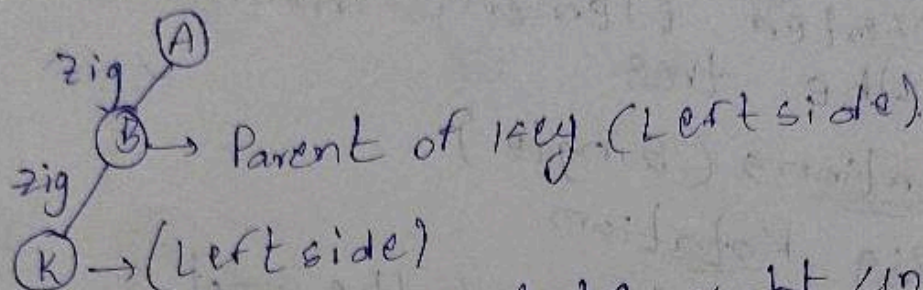
⇒ In this case rotate left.



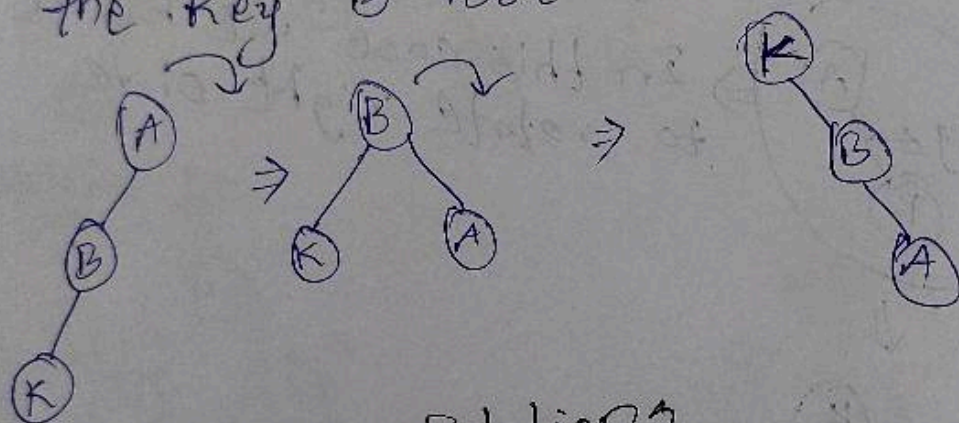
Rotations (Double)

1) zig-zig Rotation

* The parent of key is left side and key also left side.

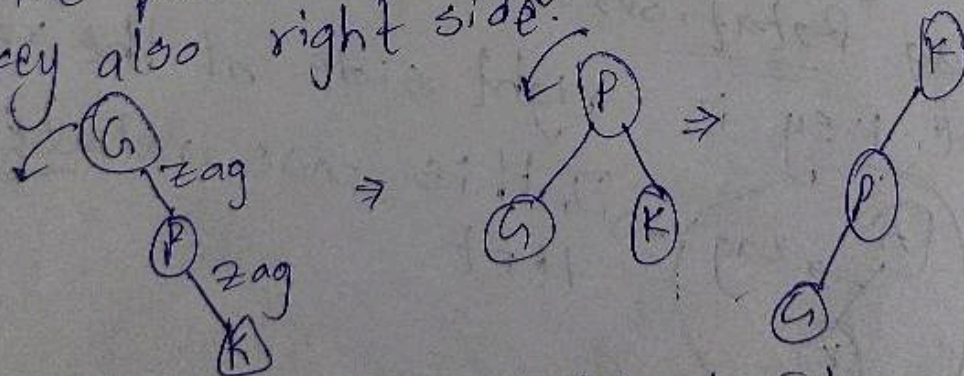


* In this case rotate right untill the key is root node.



2) zag-zag Rotations

* The parent of key is right side & key also right side.



* In this case rotate left.

3) Zig-

* The
& key

left

4) Zag

* The
& key

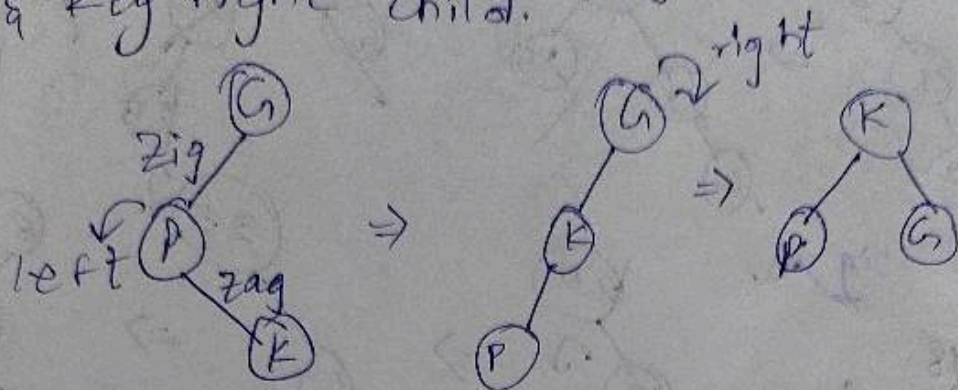
Q. To
ing

sol:- (1)

3) 25
↓

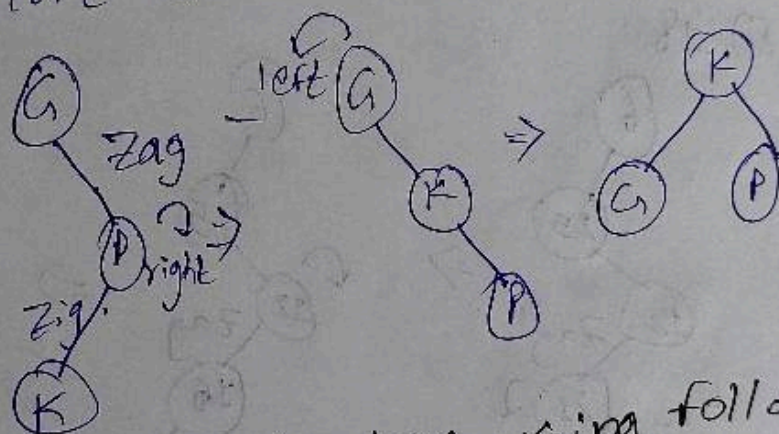
③ Zig-Zag Rotations

* The parent of key is left child & key right child.

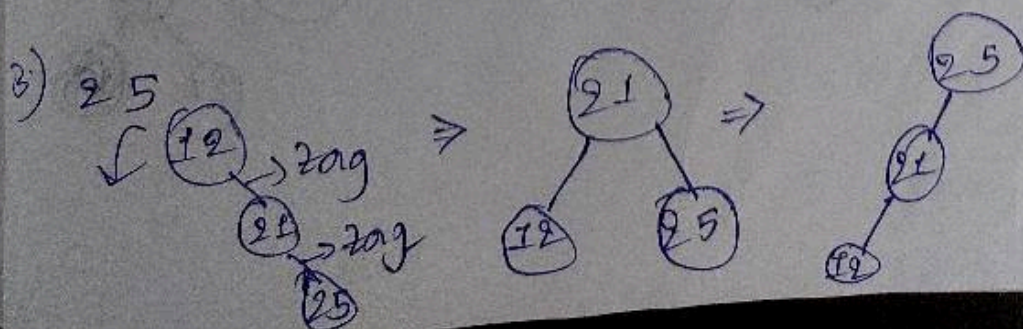
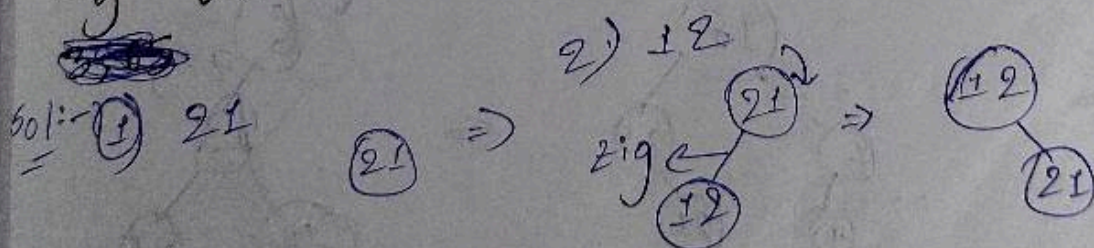


④ Zag-Zig Rotations

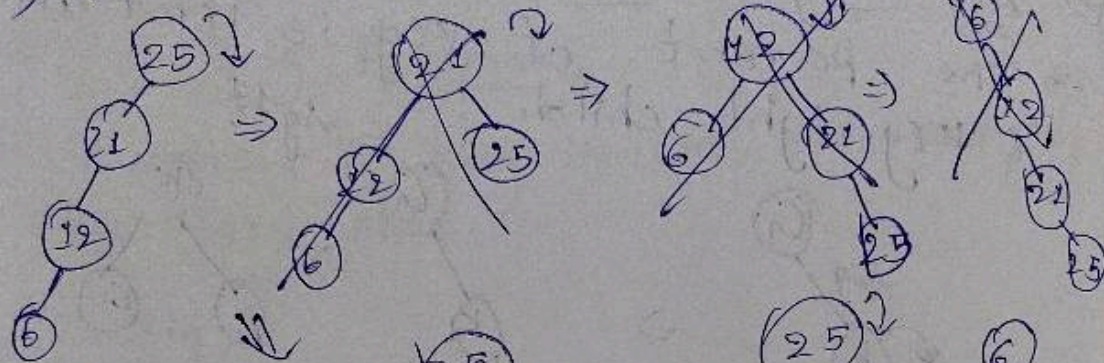
* The parent of key is right child & key is left child.



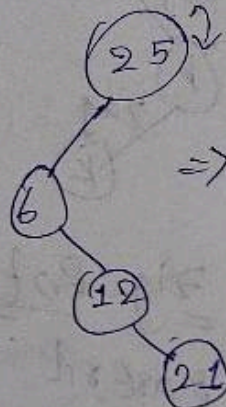
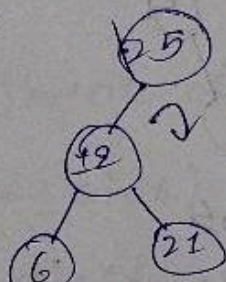
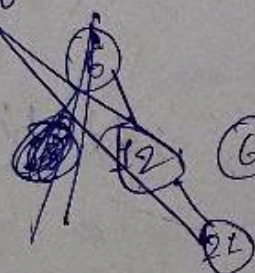
Q. To Construct a play tree using following values
21, 12, 25, 8, 18, 4, 36, 54,



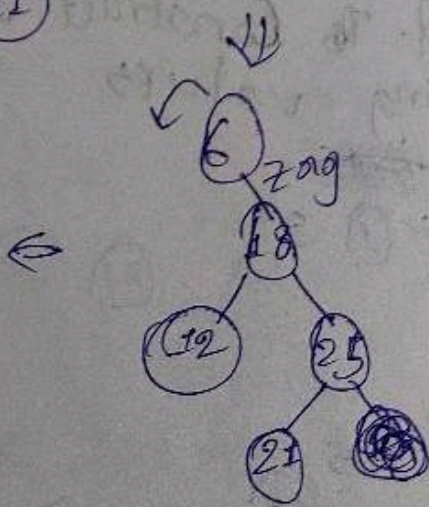
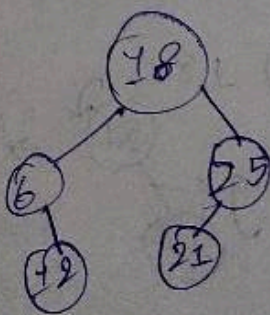
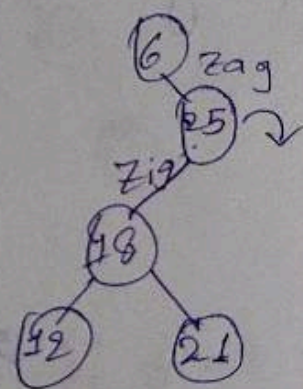
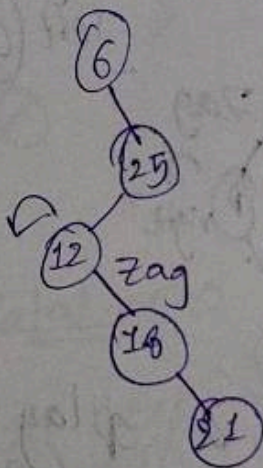
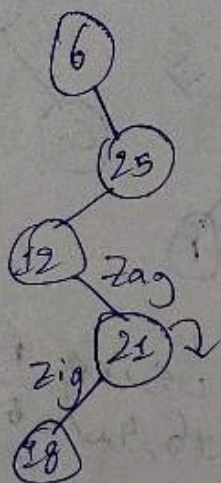
4) 6



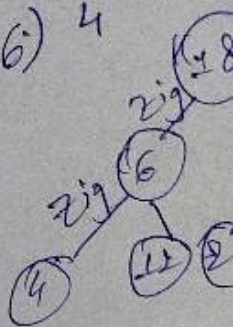
5) 18



5) 18

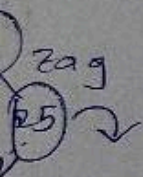
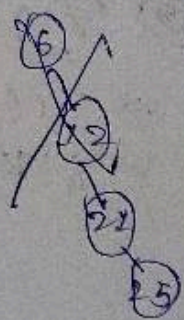


6) 4



7) 36

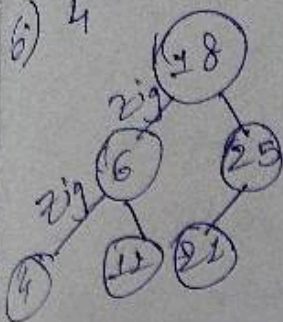
4



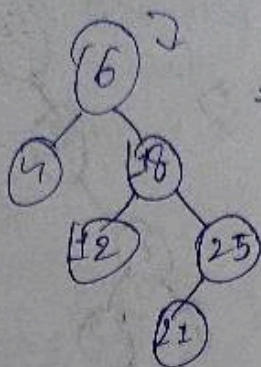
ag



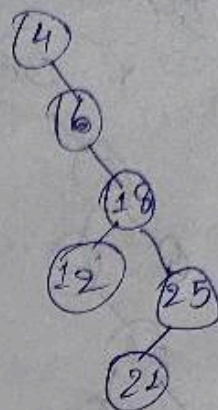
6) 4



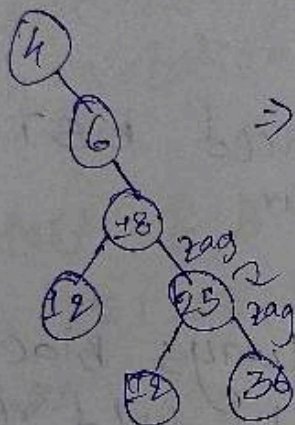
⇒



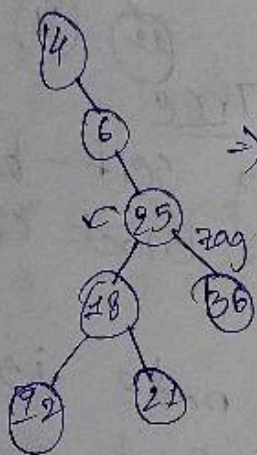
⇒



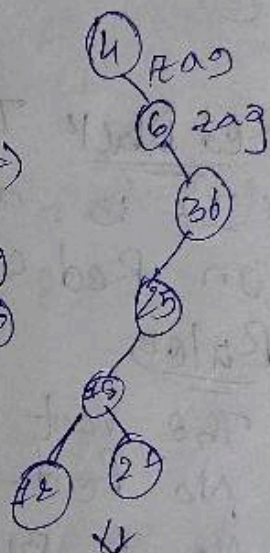
7) 36



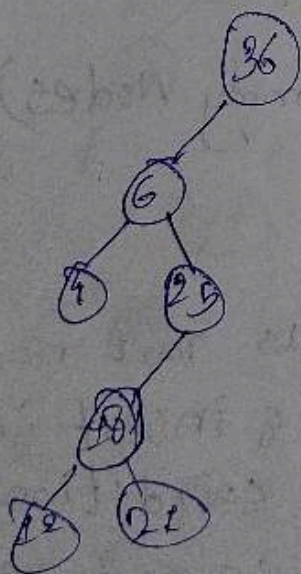
⇒



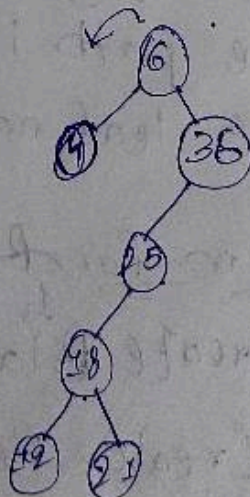
⇒



↓



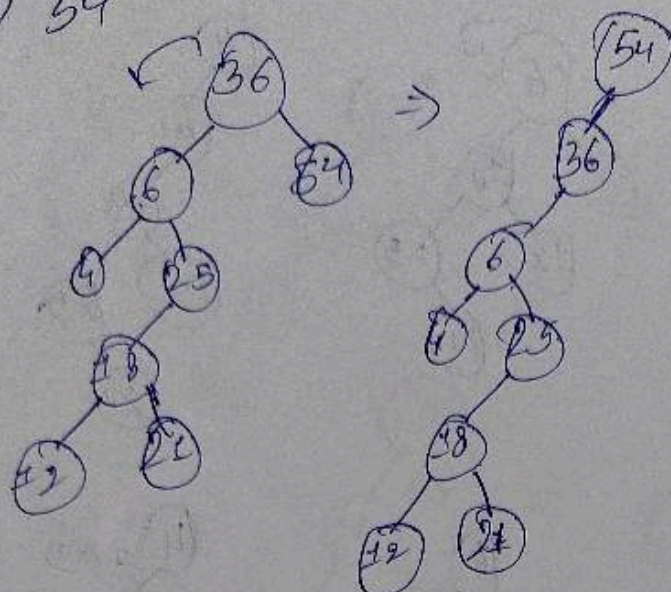
⇐



T.D

for zig-zag

8) 34



Red Black Tree

* It is a self-balanced BST based on Red & Black colour.

Rules

- ① The root node is always Black.
- ② No Red-Red parent-child relation.
- ③ No of Black nodes from root to node all the path is same.
- ④ All the leaf nodes (Dummy Nodes) are Black.

Algorithm for R.B. Tree

Step 1:- Create Black node as root node.

Step 2:- Create Red child & insert into BST using following conditions.

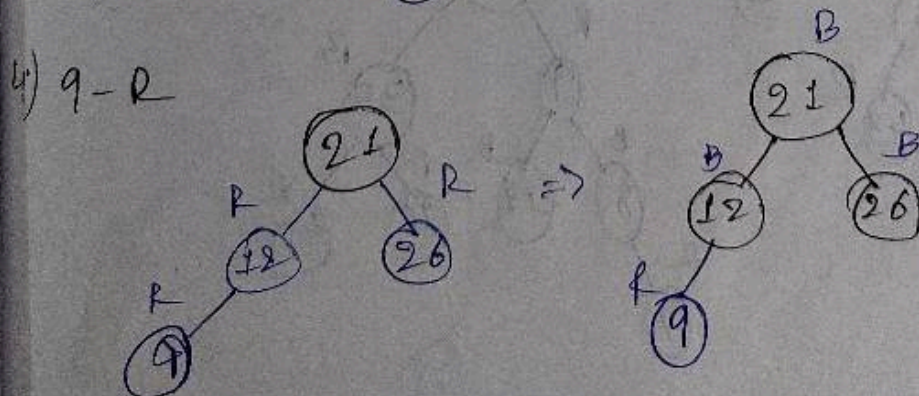
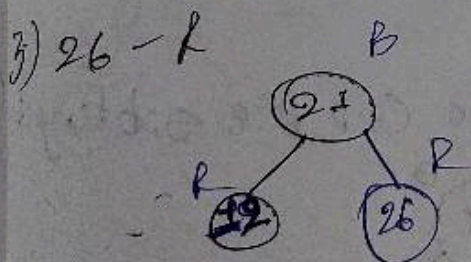
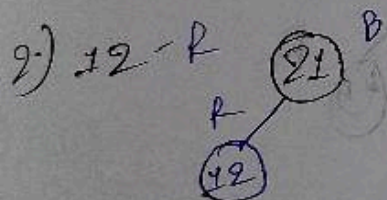
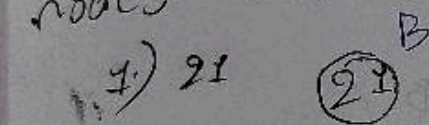
(i) If parent is Black then insert into

BST
(ii) If parent is red

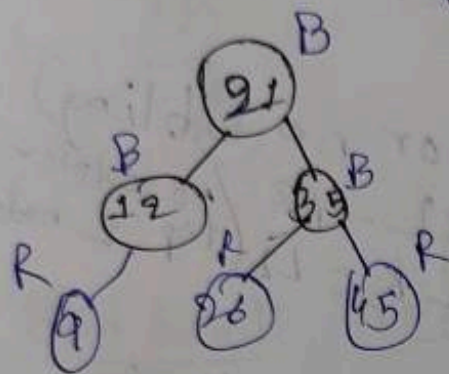
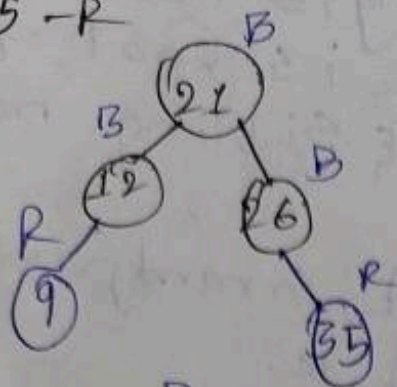
ii. 1) $\Rightarrow$ If parent sibling is red.
 ii. 1.1) $\Rightarrow$ If grandparent is root then
 recolour parent & sibling nodes.
 (otherwise)
 then recolour parent, grandparent &
 sibling nodes & recheck.

(otherwise)
 (Parent sibling is black or no sibling)
 apply rotation $\begin{matrix} LL \rightarrow RL \\ RR \rightarrow LR \end{matrix}$ (re-colour top 2 nodes)
 $RL \rightarrow RR$
 $LR \rightarrow LL$

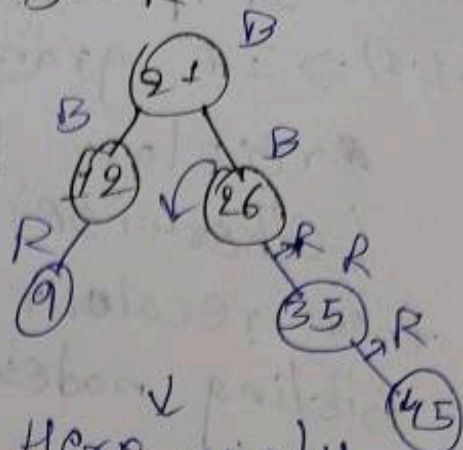
Q. Construct P.B Tree using following
 nodes. 21, 12, 26, 9, 35, 48, 8, 7, 52



5) 35 - R

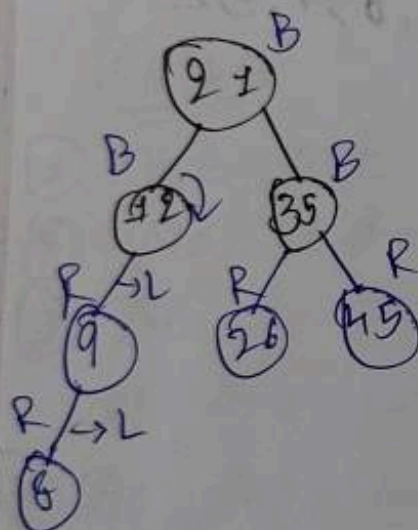


6) 45 - R

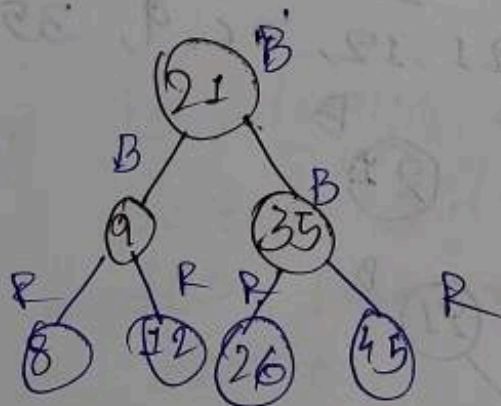


Here apply rotation as there is no sibling nodes. after rotation

7) 8 - R

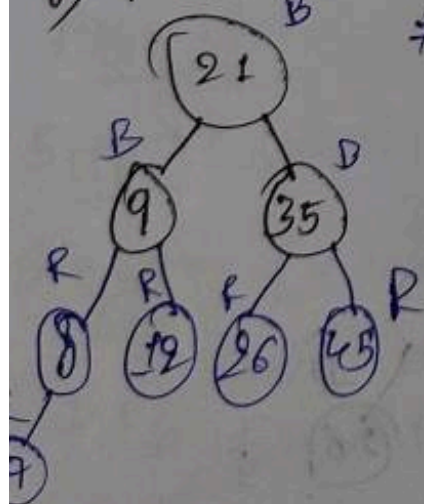


=>

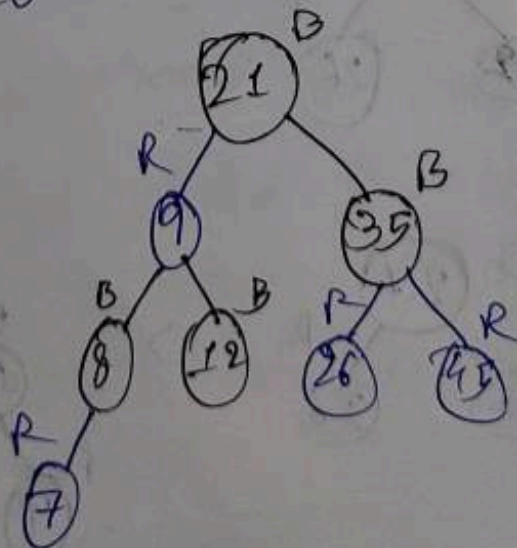


8) 7 - R

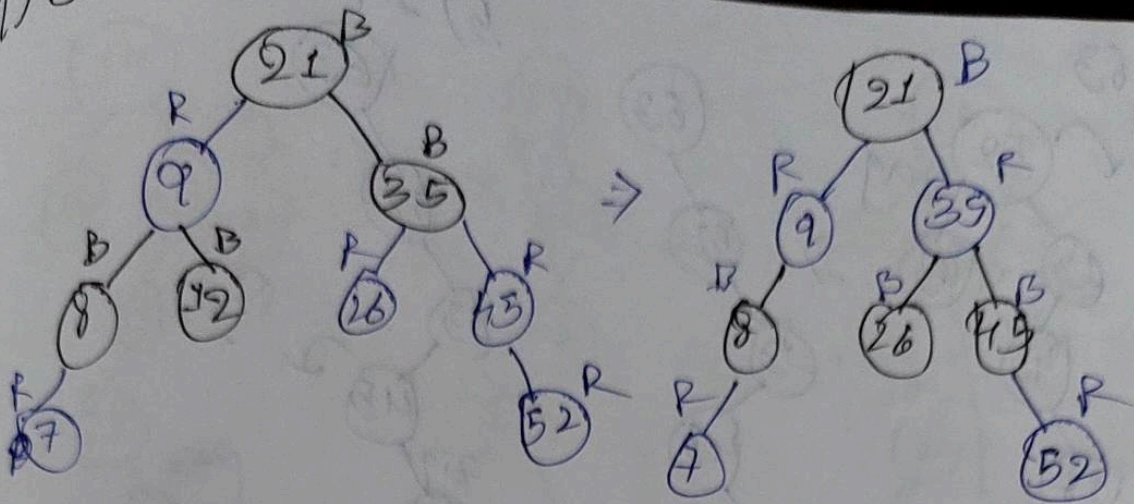
*Recolour the G.P, P & siblings



=>



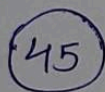
Q. 52 - R



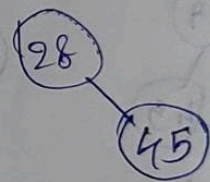
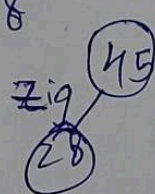
Q. 45, 28, 13, 52, 63, 14, 71 and 3. To construct splay tree.

Q. To construct Red Black Tree using following elements 6, 16, 3, 12, 15, 23, 38 & 75

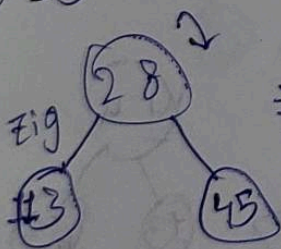
1A) 1) 45



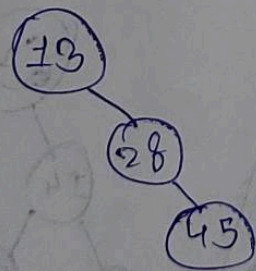
⇒ 2) 28



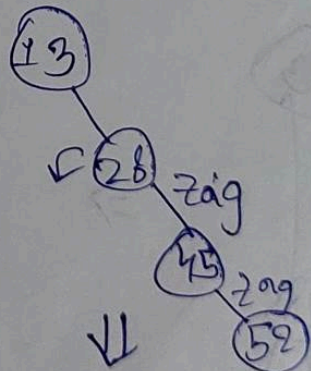
3) 13



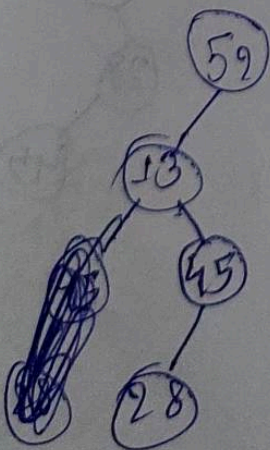
⇒



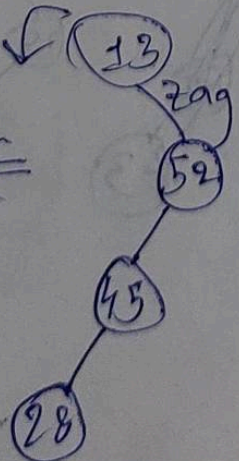
4) 52



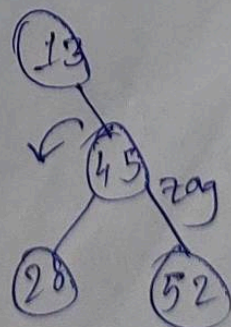
⇓



⇐



⇐



B-Tree

26-03-25

B-Tree is a height balanced m -ordered tree.

Properties

- ① In B-Tree all the leaf nodes are in same level.
- ② In B-Tree each node contain more than one key.
- ③ The keys in the nodes are in sorted order.
- ④ All the nodes (except leaf) must contain minimum $\frac{m}{2}$ and maximum m number of children.
- ⑤ In B-Tree all the nodes must contain minimum $\lceil \frac{m}{2} \rceil - 1$ and maximum $(m-1)$ keys.

Q. Construct a B-Tree using the values 1 to 10 & The order is 3.

Note:- If any node is overflow then split the node & move the middle element as parent.

1) 1

1

2) 2

1	2
---	---

3) 3

1	2	3
---	---	---

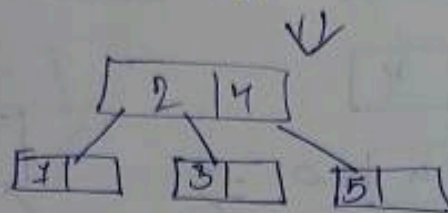
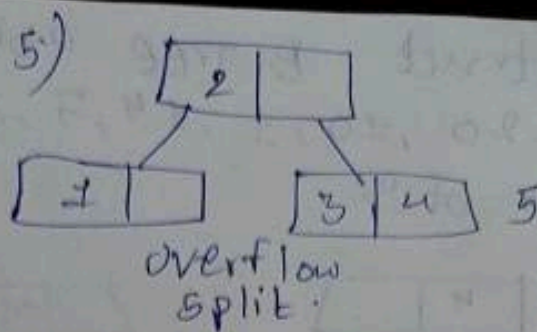
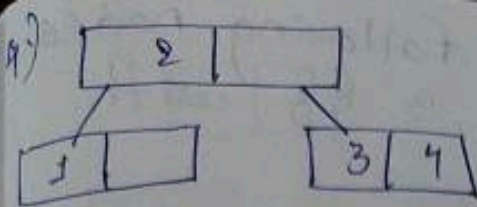
 $\Rightarrow$

2	
---	--

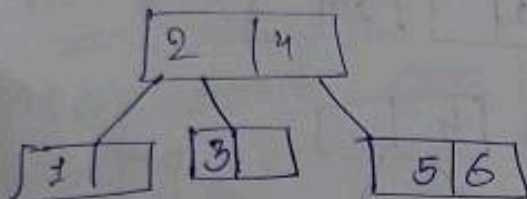
overflow then split

1	
---	--

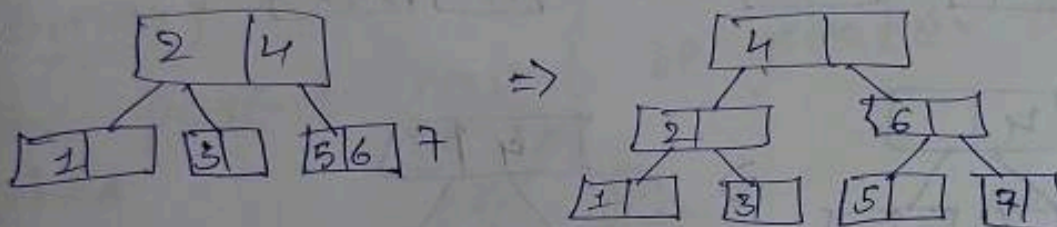
3	
---	--



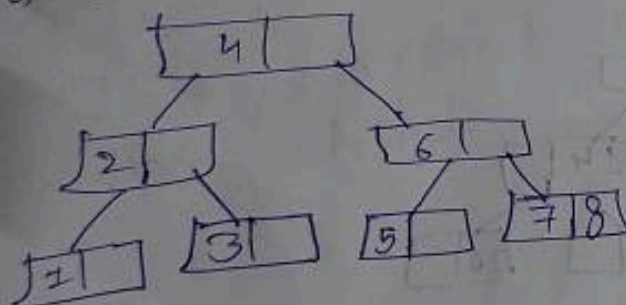
6) 6



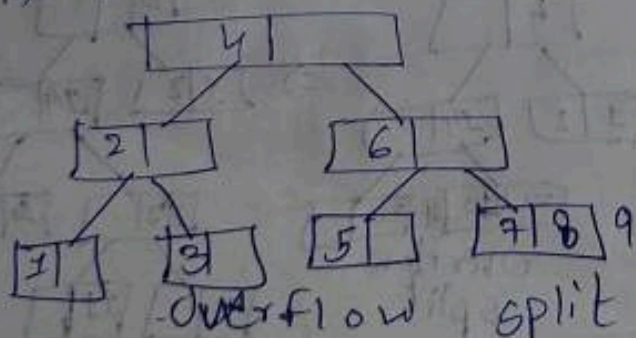
7)



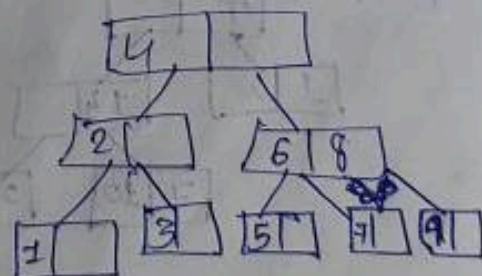
8) 8



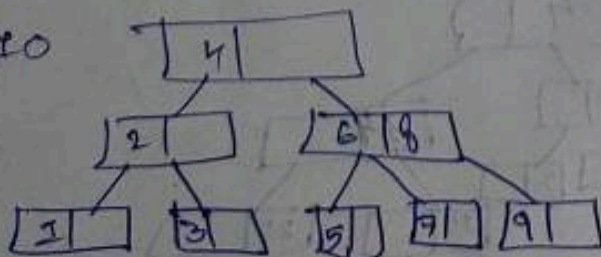
9) 9



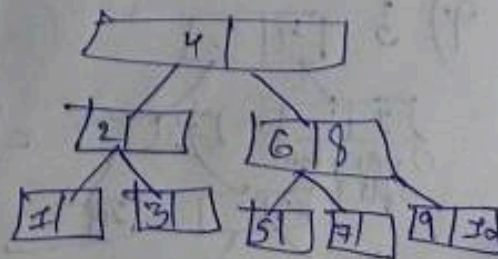
$\Rightarrow$



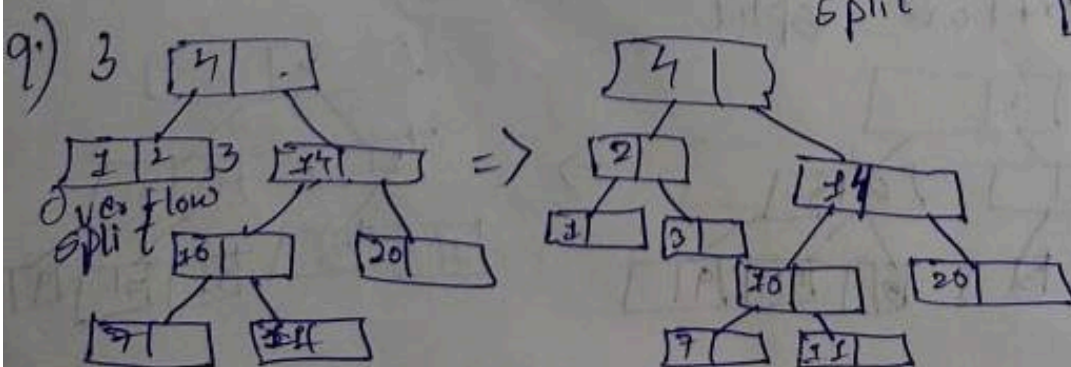
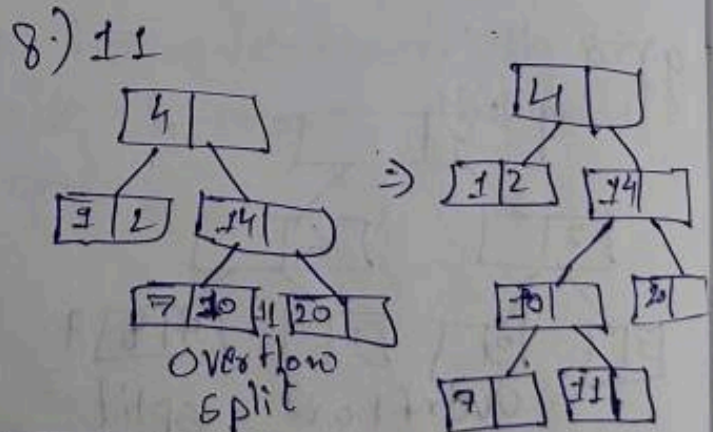
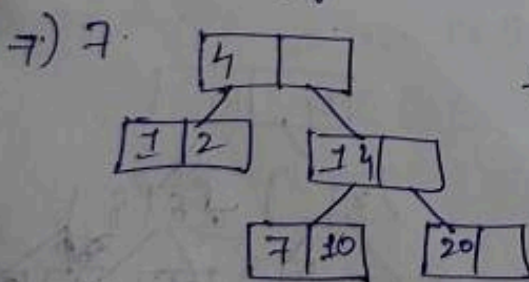
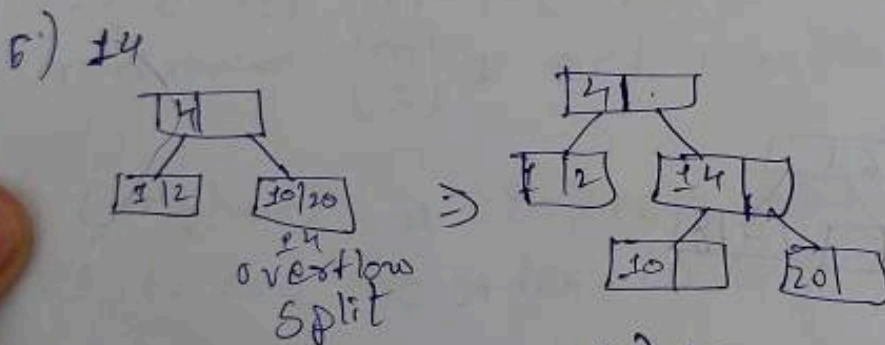
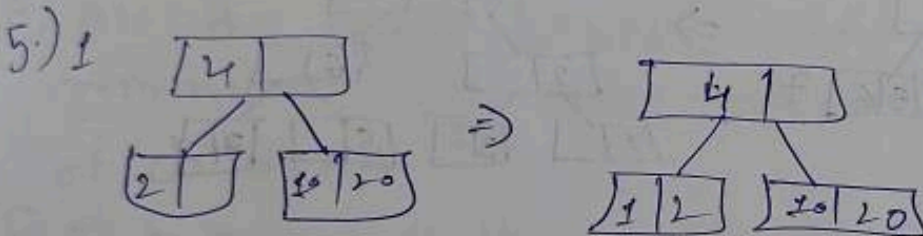
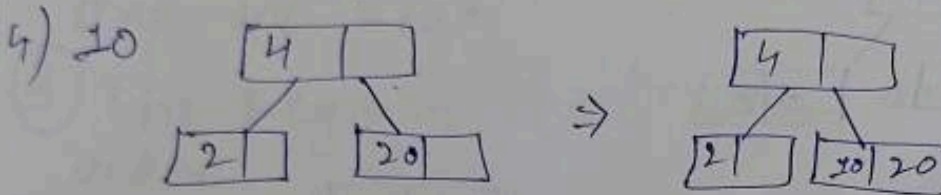
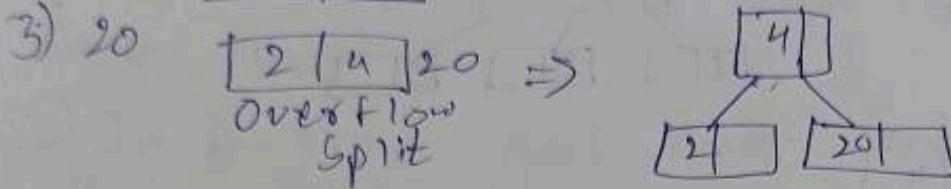
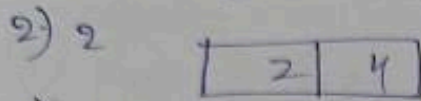
10) 10



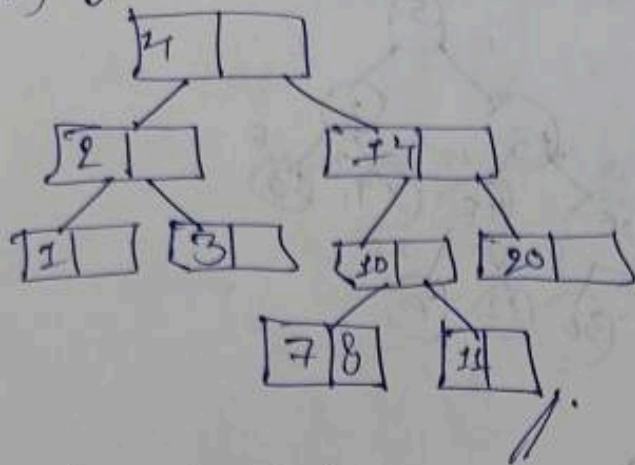
$\Rightarrow$



6. Construct B-tree using following nodes.
4, 2, 20, 10, 1, 14, 7, 11, 3 & 8. with
order of 3.



10.) 8



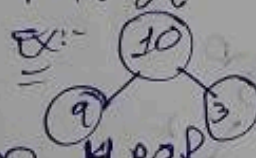
Heap Sort

01/04/25

→ Heap sort can be implemented by using binary tree.

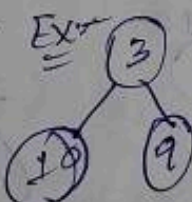
① Binary max ~~Heap~~ Heap

The parent must be greater than the child.



② Binary min Heap

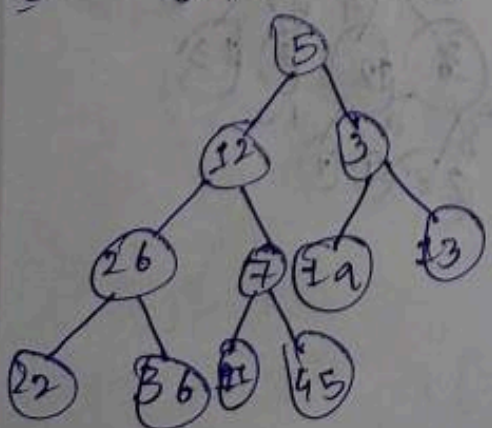
The parent must be lesser than the child.



Algorithm

Step 1:- Construct binary tree using level order.

Ex:- 5, 12, 3, 26, 7, 19, 13, 22, 36, 11, 45



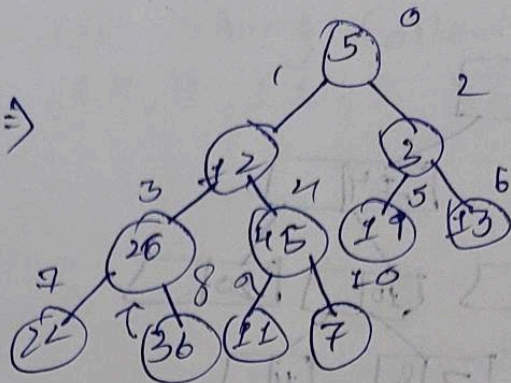
$$L = i = 4$$

$$-i = \frac{11}{2} - 1 = 4$$

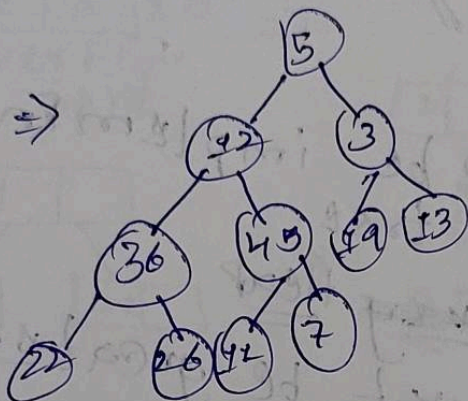
$$Lt = i * 2 + 1$$

$$Rt = i * 2 + 2$$

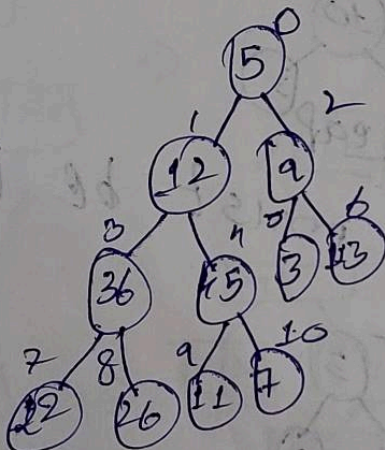
$lt > a[L]$
 $L = lt, (L = 9)$
 $rt > a[L]$
 $L = rt, (L = 10)$



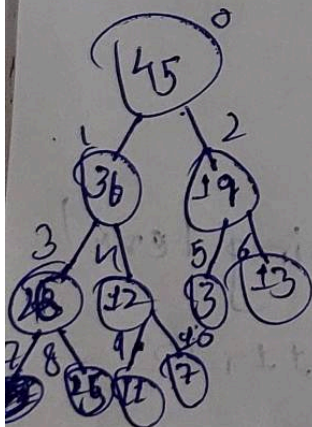
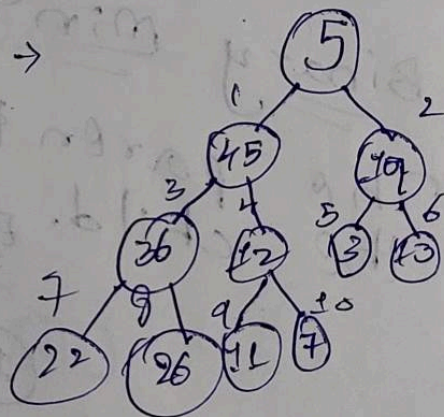
$i = 3$
 $L = 3$
 $lt = 7 \Rightarrow$
 $rt = 8$
 $L = 8$



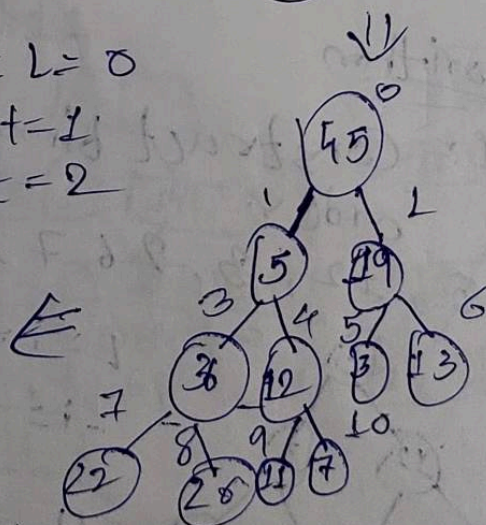
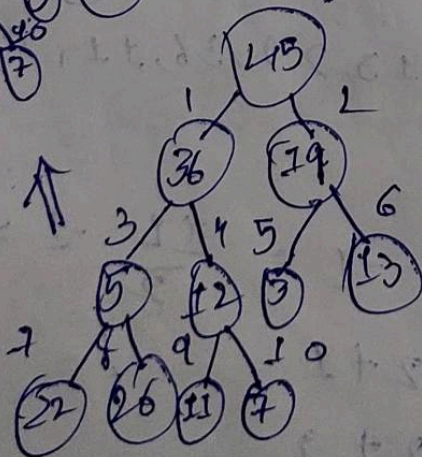
$i = 2$
 $L = 2$
 $lt = 5$
 $rt = 6$



$i = L = 4$
 $lt = 3$
 $rt = 4$

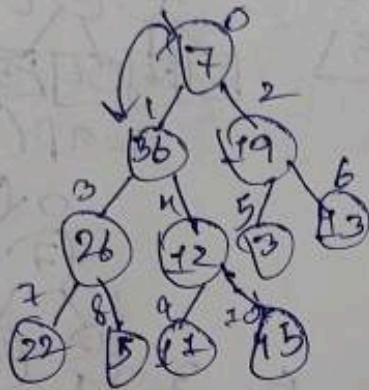


$i = L = 0$
 $lt = 1$
 $rt = 2$

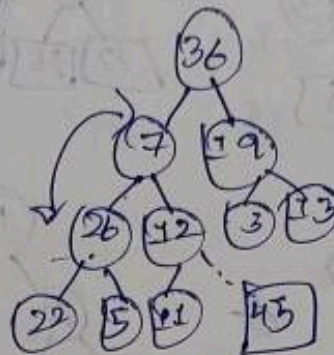


*After constructing max heap swap root and last child node.

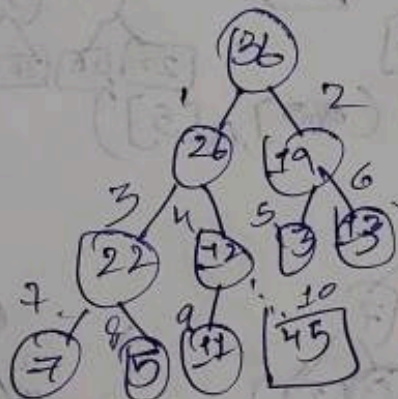
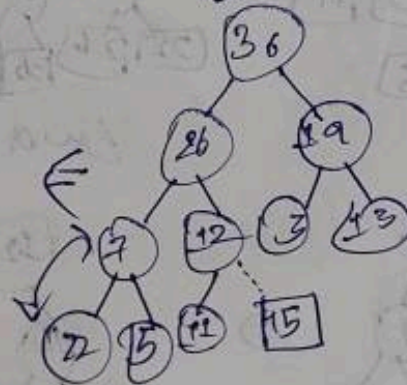
Swap ($a[0], a[10]$)



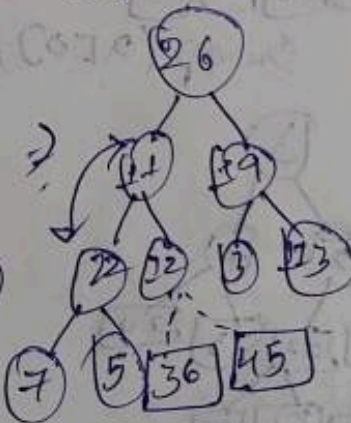
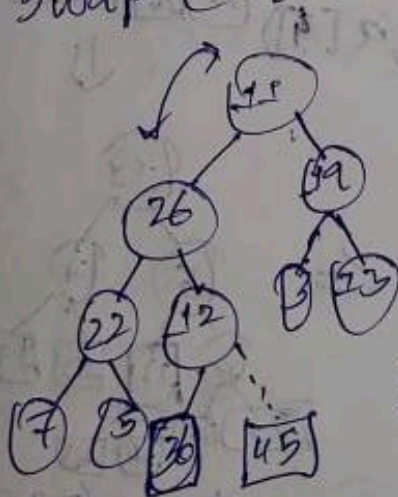
=>



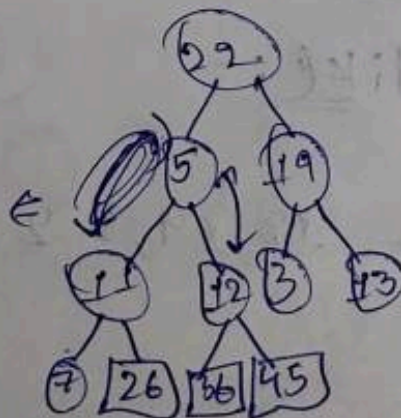
↓



Swap ($a[0], a[9]$)



Swap ($a[0], a[8]$)



Swap ($a[0], a[7]$)

Code for Heap Sort

02/04/20

```
#include <stdio.h>
```

```
void heapify(int a[], int n, int i) {
```

```
    int lt, rt, L = i, t;
```

```
    lt = 2*i + 1;
```

```
    rt = 2*i + 2;
```

```
    if (lt < n && a[lt] > a[L])
```

```
        L = lt;
```

```
    if (rt < n && a[rt] > a[L])
```

```
        L = rt;
```

```
    if (L != i) {
```

```
        t = a[i];
```

```
        a[i] = a[L];
```

```
        a[L] = t;
```

```
        heapify(a, n, L);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int a[n], i, t;
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &a[i]);
```

```
    }
```

```
    for (i = n/2 - 1; i >= 0; i--) {
```

```
        heapify(a, n, i);
```

```
    } for (i = n - 1; i >= 0; i--) {
```

```
        t = a[0];
```

```
        a[0] = a[i];
```

```
        a[i] = t;
```

```
heapify(a, i, 0);
```

```
}
```

```
for (i = 0; i < n; i++) {
```

```
    printf("%d", a[i]);
```

```
}
```

```
return 0;
```

```
}
```